

VAJE

Ukazna lupina

Vgrajeni in zunanji ukazi

- V lupini so **vgrajeni** in **zunanji** ukazi. Vgrajeni imajo prednost pri zagonu.
- `type ukaz` za ugotavljanje tipa ukaza
 - `type -a date ls if echo cd` izpiši tipe podanih ukazov
- `builtin ukaz` izvedba vgrajenega ukaza tipa ukaza
 - `builtin echo Juhuu` zagon notranjega ukaza
- Kako zagnati zunanji ukaz, če že nastopa notranji?
 - To storimo tako, da podamo dejansko pot do imenika kjer je shranjen ukaz
 - `/bin/echo Zunanji echo`
- Ukaz poženemo v trenutem imeniku tako, da dodamo predpono `./`
- Da nam predpone in celotne dejanske poti ni potrebno pisati, lahko nastavimo spremenljivko `$PATH`
 - `echo $PATH` nam da izpis imenikov, kjer lupina išče ukaze
 - `which ukaz` je izpis imenika, kjer se nahaja podani ukaz, deluje tudi za vgrajene ukaze:
 - `which ls mkdir cd cat`
 - `echo $PATH | tr : "\n" | nl` izpis imenikov iz `$PATH`
- Za olajšavo dela imamo tudi dnevnik predhodno vnešenih ukazov
 - `gor / do1` ... predhodni ukaz iz zgodovine, naslednji ukaz iz zgodovine
 - `CTRL-R` ... hitro iskanje po zgodovini
 - `CTRL-G` ... prekini iskanje po zgodovini
 - `CTRL-O` ... zaženi najdeni ukaz
 - `history` izpis ukazov v dnevniku
 - `history -c` izbris dnevnika
 - `!#` ukaz številka #
 - `!!` predhodno vnešeni ukaz
 - `!^` prvi argument predhodno vnešenega ukaza
 - `!$` zadnji argument predhodno vnešenega ukaza
 - `!*` vsi argumenti predhodno vnešenega ukaza
 - `$HISTSIZE` kapaciteta dnevnika
- Lupina omogoča tudi avtomatsko dopolnjevanje ukazov s `Tab` oz `Tab Tab`
- Posebne kombinacije tipk:
 - `CTRL-C` prekinitev programa, ki se izvaja
 - `CTRL-Z` zaustavitev programa, kasneje lahko nadaljujemo njegovo izvajanje
 - `CTRL-D` konec vnosa podatkov, npr `cat`

Delo z imeniki in datotekami

- Premikanje po imenikih:
 - `pwd` izpis trenutnega delovnega imenika
 - `echo $OLDPWD` izpis prejšnjega delovnega imenika
 - `echo $HOME` domači imenik
 - `cd ~` skok domači imenik
 - ostale `cd`-je poznamo
- Absolutna pot poteka iz korenskega imenika do datoteke, relativna pot pa poteka iz trenutnega delovnega imenika do datoteke.
- Delo z imeniki `mkdir`, `rmdir`
 - Neprazni imenik odstranimo z `rm -r`, ki rekurzivno pobriše vse elemente in podimenike.
 - Drugi ukazi: `cp`, `mv`, `rm` - ti so bolj za delo z datotekami
- Ujemanje vzorcev z imeni datotek
 - `touch foo.txt bar.txt` ustvari te datoteke

- `ls *`, `ls *.txt`, `echo b*.txt`, `echo b?`, `echo [a-c][13].*` in druge variante
- Pregled in urejanje datotek
 - `more`, `less`, `od` `octal dump`, `hexdump`
 - `vi`, `vim`, `emacs`, `nano`
- Obdelava tekstovnih datotek
 - `cat` izpis vsebine datoteke
 - `head` izpis začetnega dela
 - `tail` izpis končnega dela
 - `cut` rezanje po stolpcih
 - `sort` urejanje po vrsticah
 - `shuff` permutiranje vrstic
 - `uniq` izločanje podvojenih vrstic
 - `n1` številčenje vrstic
 - `rev` obrat vrstic
 - `tr` menjava znakov
 - `wc` štetje vrstic, besed in znakov
 - `grep` filtriranje vrstic glede na podani vzorec (regularni izraz)

Uporaba več lupin

- Poznamo lupino (Shell), ki je program, ki sprejema ukaze v obliki besedila (`bash`, `zsh` ...), in Terminal, ki pa je fizična naprava (danes ne obstajajo več), ki je povezana s fizičnim računalnikom (mainframe), nekje stran. Emulator terminala pa je aplikacija, znotraj grafičnega okola Windows, macOS, Linux, ki oponaša delovanje starega fizičnega terminala v oknu na zaslonu.
- Lupine lahko gnezdimo (torej znotraj neke lupine lahko zaženemo še eno ... itd)
 - En nivo višje se vrnemo z ukazom `exit` oz. `CTRL-D`, ki prekine trenutno lupino in zapre trenutni terminal, vrne tudi izhodno kodo.
 - Ukaz `logout` pa zapre le prijavno lupino in ne deluje v podlupinah.
 - `$SHLVL` sistemska spremenljivka nam pove trenutni nivo gnezdenja.
- Tekstovni terminal
 - `$TERM` nam pove vrsto terminala
 - `$LINES` število vrstic tekstovne mreže terminala
 - `$COLUMNS` število stolpcev tekstovne mreže terminala
 - `clear` brisanje celotnega zaslona terminala
 - `tty` ime trenutnega terminala
 - `tset` reset terminala
 - `stty` lastnosti terminala

Skriptiranje

Skripta

- Zgradba skripte
 - `#!` oz Shebang, `#!/bin/bash` je začetna vrstica shell skripte
 - `#` komentar
 - Ukaze v skripti ločujemo s podpičji ali skoki v naslednjo vrstico
 - Skripto zaključimo implicitno ali z ukazom `exit STATUS`
- Izhodni status
 - Izhodni status je 8 bitna vrednost 0 do 255
 - Spremenljivka `$?` nam pove izhodni status skripte / ukaza, ima vrednost 0 če je zaključek uspešen
 - Če je vrednost > 0, potem vedno predstavlja kodo napake.
- Zagon skripte
 - `./skripta.sh`, pravice za izvajanje dodamo z ukazom `chmod +x`, ki spremeni pravice uporabnika, da lahko to skripto poganja (x pomeni execute)
 - Skripto lahko uporabimo tudi kot knjižnico v drugi shell datoteki: `source skripta.sh`

Spremenljivke

- Argumenti skripte

- `##` je število vseh argumentov
- `$0` ime skriptne datoteke
- `$1`, `$2` posamezni argumenti skripte
- `$*`, `$@` vsi argumenti skripte
- primer:

```
#!/bin/bash
echo "Ime skripte: $0"
echo $1
echo $1$2
# 12. argument
echo $12 # previdno
echo ${12} # pravilno
```

- Čez argumente skripte lahko enostavno prehajamo z ukazom `shift`, ki premakne trenutni prvi argument:

```
while [[ -n $1 ]]; do
    echo $1
    shift
done
```

- Globalne in lokalne spremenljivke

- globalne spremenljivke: `ime=vrednost` Pozor! brez presledka ker je glupo narejen jezik.
- lokalne spremenljivke: `local ime=vrednost`

- Enojne in dvojne navenice

- enojne navednice: dobesedna predstavitev niza
- dvojne navednice: izvedejo se morebitne notranje substitucije
- uporaba navednic je nujna, kadar niz vsebuje presledke in je skoraj vedno priporočljiva
- primer:

```
a=Triglav
echo $a #vrne Triglav
echo '$a' #vrne $a
echo "$a" #vrne Triglav
b='Martin      Krpan'
echo $b #vrne Martin Krpan
echo '$b' #vrne $b
echo "$b" #vrne Martin      Krpan
```

- Substitucija vrednosti spremenljivk

- kot že vemo, spremenljivko priključimo s `$ime` oz `${ime}` oz `${ime:-privzeto}`, zadnja vrne shranjeno ali privzeto vrednost
- `$` lahko uporabljamo kot substitucijo, primer:

```
cmd=ls
arg=-alp
echo Ukaz: $cmd z argumenti $arg
```

- Lahko odstranjujemo podnize, jih zamenjujemo, glej *man bash*:

- Nizi in podnizi

- `${#ime}` ... dolžina niza, shranjenega v ime
- `${ime:poz}` ... podniz od indeksa poz do konca
- `${ime:poz:dol}` ... podniz od indeksa poz dolžine dol

- Odstranjevanje podnizov

- `${ime#vzorec}` ... najkrajša predpona
- `${ime##vzorec}` ... najdaljša predpona
- `${ime%vzorec}` ... najkrajša pripona
- `${ime%%vzorec}` ... najdaljša pripona

- Zamenjava podniza

- `${ime/vzorec/vrednost}` ... prva pojavitev
- `${ime//vzorec/vrednost}` ... vse pojavitve

Pogojni izrazi

- Ovrednotenje pogoja
 - več možnosti `test pogoj`, `[ pogoj ]`, `[[ pogoj ]]`
 - `test 05 == 05`; `echo $?` rezultat dobimo preko izhodnega statusa
 - `[[ 3 -lt 42 ]]`; `echo $?` rezultat dobimo preko izhodnega statusa
- Primerjava nizov
 - `[[ -z niz ]]` je niz prazen (dolžina 0, zero)
 - `[[ -n niz ]]` Je niz neprazen (dolžina >0, nozero)?
 - `[[ niz1 = niz2 ]]` Sta niza enaka?
 - `[[ niz1 != niz2 ]]` Sta niza različna?
 - `[[ niz1 < niz2 ]]` Je niz1 leksikografsko pred niz2?
 - `[[ niz1 > niz2 ]]` Je niz1 leksikografsko za niz2?
 - mnogih težav se rešimo z uporabo navednic
- Preverjanje datotek
 - `[[ -e pot ]]` Ali datoteka obstaja?
 - `[[ -f pot ]]` Je datoteka navadna (regular file)?
 - `[[ -d pot ]]` Je datoteka imenik?
 - `[[ -L pot ]]` Je datoteka simbolična povezava?
 - `[[ -b pot ]]` Je datoteka bločna naprava?
 - `[[ -c pot ]]` Je datoteka znakovna naprava?
 - `test -e /etc/passwd` primer
- Logični izrazi
 - `[[ -n "$x" && -f /etc/passwd ]]` Logični IN
 - `[[ -f "$f" || test -L "$f" ]]` Logični ALI
 - `[[ ! -f /etc/passwd ]]` Negacija
 - lahko tudi takole: `test ! -f /etc/passwd`
- Logična alternativa
 - `ukaz && ukaz`
 - `ukaz || ukaz`
 - vrednotenje izraza je kratkostično (short circuit)
 - oba sta enako prioritete
 - `true || echo "To pa se ne izpiše"`
 - `false || echo "To pa se"`
- Aritmetični pogoji
 - `[ a -eq b ]` Je a enako b?
 - `[ a -ne b ]` Je a različno od b?
 - `[ a -gt b ]` Je a večje od b?
 - `[ a -ge b ]` Je a večje ali enako od b?
 - `[ a -lt b ]` Je a manjše od b?
 - `[ a -le b ]` Je a manjše ali enako od b?
 - `test 5 -eq 500` kot primer s testom
- Dvojni oklepaji:
 - Cjevsko izrazoslovje, ovrednotenje izraza `(( izraz ))`, in substitucija `$(( izraz ))`
 - primer:

```
(( sedem = 1 + 2 * 3 ))
sedem=$(( 1 + 2 * 3 ))
(( a = 1, a++, a += 2, b = a + a, c = a*b ))
(( a <= 42 )) && echo Malo || echo Veliko
echo $a $(( 2 * a + b ))
```

Programski stavki

- Odločitveni stavek if

```
if pogoj; then ukazi; fi
if pogoj; then ukazi; else ukazi; fi
if pogoj; then ukazi; elif pogoj; then ukazi; fi
if pogoj; then ukazi; elif pogoj; then ukazi; else ukazi; fi
```

- Zanke

- while zanka:

```
while pogoj; do ukazi; done
until pogoj; do ukazi; done
```

- seveda omogoča tudi `break` in `continue`

- for zanka:

```
for var in spisek; do ukazi; done
for ((init; pogoj; inkrement)); do ukazi; done
```

```
seznam='a b c d'
for x in $seznam; do echo $x; done
for x in "$seznam"; do echo $x; done
for x in '$seznam'; do echo $x; done
for x in $seznam $seznam $seznam; do echo $x; done
for (( i = 0; i <= 42; i++ )); do echo $i; done
```

- Stavek case:

```
case niz in
    (vzorec1) ukazi ;;
    (vzorec2) ukazi ;;
    (*) ukazi ;;
esac
```

```
akcija=$1
shift
case $akcija in
    (help) do_help ;;
    (list_all) ls -R "$1" ;;
    (update) shift; do_update $* ;;
    (*) echo Napačna akcija.
esac
```

- Ujemanje vzorcev z nizi (to niso regularni izrazi):

- `\*` poljuben niz, vključno s praznim nizom
- `?` poljuben znak
- `[...]` poljuben znak znotraj oglatih oklepajev
- ostalo: znak predstavlja sam sebe
- primer:

```
case $niz in
    bingo) echo 1 ;;
    bin*) echo 2 ;;
    [abc]ingo) echo 3 ;;
    ???go) echo 4 ;;
    ??) echo 5 ;;
    *) echo 6 ;;
esac
```

Funkcije

Izvajanje v podlupini

- Podlupina ja podproces trenutne lupine, nudi avtomatsko dedovanje spremenljivk, funkcij, odprtih datotek, ne deduje pa rokovalnikov signalov.
- Osnovni zagon podlupine: `( ukaz )`
 - primeri:

```
( hostname )

( ( ( echo -n $BASH_SUBSHELL ) ); echo $BASH_SUBSHELL ) )

( echo Zivjo; exit 42; echo svet. )

( echo ZACETEK; ls -alp; echo KONEC )

pwd; ( cd /; pwd ); pwd
```

- Zagon podlupine s substitucijo: `$( ukaz )`
 - primeri:

```
$( hostname )
h=$(hostname)

dirname $(which less)

u=$(whoami); h=$(hostname); echo Dobrodošel $u na $h
echo Dobrodošel $(whoami) na $(hostname)

data=$(cat /etc/passwd)

echo $(echo $(echo $(echo $(echo $(echo $RANDOM))))))
```

- Dedovanje spremenljivk:
 - primer:

```
#Lupina
x=42; echo $x
# Podlupina
( echo $x; x=1; echo $x )
# Spet lupina
echo $x
```

Ukazni bloki

- Združevanje ukazov v blok
 - `{ ukazi; }` pozor na presledek za `{ in ; pred }`.
 - primer združevanja pogojev:

```
# Združevanje pogojev
true && { false || true; }
# Združevanje ukazov
mkdir test && { cd test; touch dat.txt dat.jpg; }
[[ $(whoami) == root ]] && { echo "I'm the boss."; cat /etc/shadow; }
```

- Ponovitev

```
{ pwd; ls; } # združevanje ukazov v blok
{a,b,c} # posebna razširitev
( pwd ) # podlupina
(( a=1+2 )) # aritmetika
[ a = b ] # pogojni izrazi
[[ -d /bin ]] # razširjeni pogojni izrazi
```

Funkcije

- Definicija funkcij
 - definicija, pozor!, telo funkcije je blok, zato veljajo enaka pravila

```
function ime {ukazi; } #bolj bashovsko
ime() { ukazi; } # bolj prenosljivo
```

- primer, vsi primeri so enakovredni:

```
function f { echo F; }  
function f {  
    echo F  
}  
f() { echo F; }  
f() {  
    echo F  
}
```

- Uporaba funkcij

- funkcije kličemo enako kot vse ostale ukaze:

```
function f { echo F; }  
f  
  
function do_big_thing {  
    cat /etc/passwd | cut -d: -f7 | sort | uniq -c  
}  
do_big_thing  
  
klici=f  
$klici
```

- funkcije lahko tudi dedujemo:

```
# Funkcija definirana v lupini  
function f { echo F; }  
f  
( f )  
  
# Funkcija definirana v podlupini  
(  
    function g { echo G; }  
    g  
)  
g
```

- argumenti funkcije delujejo na enak način kot argumenti skripte; spremenljivke: \$1, \$2, ...

```
function fullname {  
    local name=${1:-Janez}  
    local surname=${2:-Novak}  
    echo $name $surname  
}  
  
function print_my_args {  
    while [[ -n $1 ]]; do  
        echo $1  
        shift  
    done  
}
```

- rezultat funkcije: Lahko uporabimo izhodni status, ampak to deluje za števila, ne pa za nize.

```
# Če vrnemo kot izhodni status  
function vsota_fail {  
    local vsota=0  
    while [[ -n $1 ]]; do  
        (( vsota += $1 ))  
        shift  
    done  
    return $vsota  
}
```

- o Bolje je pa če za to uporabimo standardni izhod: `return` nadomestimo z `echo`, rezultat pa ujamemo preko izvajanja v podlupini, izhodni status pa uporabimo za sporočanje uspešnosti izvedbe funkcije:

```
function vsota {
    local vsota=0
    while [[ -n $1 ]]; do
        (( vsota += $1 ))
        shift
    done
    echo $vsota
    return 0
}
```

Preusmerjanje

Nadzor poslov

- Zagon ukaza v ospredju - običajen zagon
 - o proces ki nastane je otrok lupine:
 - se zažene in je povezan s terminalom
 - vrne izhodni status, ki ga lupina prevzame, spremenljivka `$?`.
 - o lupina, ki je starš zagnanega procesa:
 - čaka da se otrok konča, nato prevzame njegov izhodni status
- Zagon ukaza v ozadju z `&`
 - o proces ki nastane je otrok lupine:
 - se zažene in njegov *stdout* je povezan s terminalom
 - vrne izhodni status, ki ga lupina prevzame
 - o lupina, ki je starš zagnanega procesa
 - **ne** čaka da se otrok konča
 - prevzame njegov izhodni status, ko ji OS to sporoči preko signala `SIGCHLD`
 - spremenljivka `$?` hrani *pid* zagnanega otroka
 - primer:

```
firefox &
echo $!
```

- o koliko časa trajajo ta izvajanja

```
sleep 1 ; sleep 2 ; sleep 3 # traja 6 sekund, vse v ospredju
sleep 1 & sleep 2 & sleep 3 & # traja 0 sekund, vse v ozadju
sleep 1 & sleep 2 ; sleep 3 & # traja 2 sekundi, sleep 1 in sleep 3 v ozadju
```

- Posel (job)
 - o je opravilo, ki je v celoti pripravljeno izvajanje
 - vključno z vhodnimi podatki, ki so običajno pripravljene v datotekah
 - brez posebne interakcije z uporabnikom
 - o posli v Bash
 - so koncepti lupine (ne OS)
 - ukazi zagnani v okviru lupine so posli
 - običajno posel zaženemo v ozadju
 - `JID` posla je njegova številka (*job id*)
- Vgrajeni ukazi za delo s posli
 - o `jobs` izpis trenutno tekočih poslov
 - o `CTRL-Z` zaustavitev posla v ospredju
 - o `fg jid` nadaljevanje posla v ospredju
 - o `bg jid` nadaljevanje posla v ozadju
 - o primer:


```
sleep 10 & sleep 20 & sleep 30 &  
jobs  
fg
```

- Ko se lupina konča, pokonča svoje posle, pošlje jim signal `SIGHUP`
- Ločitev posla od lupine, na način ga ne pokonča.

To storimo z ukazom `disown`.

- Omejitev procesa
 - To lahko storimo z ukazom `ulimit`

```
ulimit -a  
-t: cpu time (seconds) unlimited  
-f: file size (blocks) unlimited  
-d: data seg size (kbytes) unlimited  
-s: stack size (kbytes) 8176  
-c: core file size (blocks) 0  
-v: address space (kbytes) unlimited  
-l: locked-in-memory size (kbytes) unlimited  
-u: processes 2666  
-n: file descriptors 256
```

Preusmerjanje

Osnovno preusmerjanje

- Ukazi
 - `ukaz < datoteka` preusmerjanje *stdin*
 - `ukaz > datoteka` preusmerjanje *stdout*, prepisovanje (*overwrite*)
 - `ukaz >> datoteka` preusmerjanje *stdout*, dodajanje (*append*)
 - primer:

```
cat <mouse.txt  
cat mouse.txt ... enak učinek  
  
cat </dev/urandom  
  
read priv pub <secret.keys  
  
cat </etc/shadow >~/gesla-leak
```

Splošno preusmerjanje

- Ukazi
 - `ukaz deskriptor < datoteka`
 - `ukaz deskriptor > datoteka`
 - `ukaz deskriptor >> datoteka`
 - `ukaz deskriptor1 >& deskriptor2`
 - `ukaz 2> datoteka` samo *stderr*
 - `ukaz 1> datoteka 2>&1` *stderr* in *stdout*
 - `ukaz &> datoteka` okrajšava
 - primer:

```
mkdir /test 2>/dev/null  
  
ls -R ~ >moji_fajli 2>moje_tezave  
ls -R ~ &>vse_skupaj
```

- Tukaj dokument (*here document*)
 - pravili:

```
ukaz <<EOF
... dokument, izvedba substitucij
... echo $1 in \"$1
EOF
```

```
ukaz <<'EOF'
... dobesedni dokument
... echo $1
EOF
```

- primer:

```
read -p 'Vnesi ime: ' ime
cat <<EOF >skripta.sh
#!/bin/bash
echo Živjo $ime.
EOF
```

- Tukaj niz (*here string*)

- pravilo:

```
ukaz <<< niz
```

- primeri:

```
read a b c <<< '1 2 3'
```

```
tr '[a-z]' '[A-Z]' <<< velike-crke
```

```
tr '13579' '02468' <<< $(seq 0 9)
```

- Preusmerjanje

- pravilo:

```
ukaz1 | ukaz2 | . . . | ukazN
```

- primeri:

```
cat /etc/services | egrep '/tcp' | egrep '42'
cat /etc/shells | egrep '^/bin/' | cut -d/ -f3 | tr "\n" ","
cat /etc/passwd | cut -d: -f7 | sort | uniq -c | sort -gr | head -3 | nl
cat /etc/shells | sort | tee lupine | nl
```

- Primeri:

- Enakovrednost ukazov in funkcij

```
function my_nl {
    local count=${1:-1}
    while read line; do
        echo "$count: $line"
        (( count++ ))
    done
}

cat /etc/passwd | my_nl 42

cat /etc/passwd | nl -i 2
```

- Branje datotek:

```
while read line; do
    echo $line
done
```

Jedro in sistemski klici

Dostop do jedra OS

- Preko nekaterih ukazov
 - `uname` izpis informacij o jedru in OS
 - `arch` izpis arhitekture procesorja
 - `hostname` izpis imena gostiteljskega računalnika
 - `free` izpis informacije o pomnilniku
 - `dmesg` izpis sistemskih obvestil
 - `sysctl - a` izpis sistemskih nastavitev
- Preko `/proc`
 - `man proc`, in pregled datotek: `version`, `cmdline`, `uptime`, `modules`, `cpuinfo`, `meminfo`, `diskstats`, `device`, `dma` ...
 - napiši skripto za izpis sistemskih informacij:

```
#!/bin/bash

echo Ime računalnika: $(cat /etc/hostname)
echo

echo 'Operacijski sistem: '
cat /proc/version
echo

echo Št. jedr procesorja: $(egrep processor /proc/cpuinfo | wc -l)
echo

echo Velikost pomnilnika: $(egrep MemTotal /proc/meminfo | cut -d: -f2)
echo

echo Št. procesov $(echo /proc/[0-9]* | wc -w)
echo
```

- Preko sistemskih klicev (to je edini in pravi oz neposredni način)
 - Sistemske klice lahko izvajamo:
 - v zbirniku
 - preko ovojnih funkcij
 - preko funkcije `syscall(...)`
 - posredno preko ostalih funkcij
 - Ovojne funkcije sistemskih klicev se nahajajo v C knjižnici

```
#include <unistd.h>
```

- Pregled sistemskih klicev:
 - datoteke:

```
open()    // odpre datoteko
close()   // zapre datoteko
read()    // prebere podatke iz datoteke
write()   // zapiše podatke v datoteko
lseek()   // premakne offset odprtega datotečnega deskriptorja
fcntl()   // izvede različne operacije na odprtem datotečnem deskriptorju
dup()     // podvoji datotečni deskriptor; nov deskriptor dobi najnižjo prosto številko
dup2()    // podvoji datotečni deskriptor na podano številko
```

- datoteke2:

```
rename() // preimenuje datoteko ali imenik
link()   // ustvari trdo povezavo (hard link) do datoteke
unlink() // odstrani ime datoteke (zbriše povezavo; datoteka se izbriše, ko ni več povezav)
chmod()  // spremeni dostopne pravice datoteke
chown()  // spremeni lastnika in/ali skupino datoteke
stat()   // pridobi informacije o datoteki (velikost, pravice, časovne oznake ...)
access() // preveri, ali ima proces zahtevane pravice za dostop do datoteke
```

- imeniki:

```
chdir() // spremeni trenutni delovni imenik
mkdir() // ustvari nov imenik
rmdir() // odstrani prazen imenik
symlink() // ustvari simbolno povezavo (symbolic link)
readlink() // prebere cilj simbolne povezave
```

- zaščita:

```
getuid() // vrne uporabniški ID (real UID)
setuid() // nastavi uporabniški ID procesa
getgid() // vrne ID skupine (real GID)
setgid() // nastavi ID skupine procesa
geteuid() // vrne efektivni uporabniški ID
getegid() // vrne efektivni ID skupine
umask() // nastavi masko privzetih dovoljenj za nove datoteke
```

- procesi:

```
fork() // ustvari nov proces kot kopijo trenutnega procesa
execve() // zamenja trenutno izvajani program z novim
wait() // počaka na zaključek podrejenega procesa
waitpid() // počaka na določen podrejeni proces
exit() // zaključi izvajanje procesa
nice() // spremeni prioriteto (razred razporejanja) procesa
```

- razno:

```
getpid() // vrne ID trenutnega procesa
getppid() // vrne ID nadrejenega procesa
time() // vrne trenutni sistemski čas
sleep() // začasno ustavi izvajanje procesa za podano število sekund
```

- naprave:

```
mount() // priklopi datotečni sistem na izbrano priklopno točko
unmount() // odklopi (odmontira) datotečni sistem
```

- medprocesna komunikacija:

```
pipe() // ustvari enosmerni komunikacijski kanal med procesi
mmap() // preslika datoteko ali pomnilnik v naslovni prostor procesa
munmap() // odstrani preslikavo, ustvarjeno z mmap()
shmget() // ustvari ali pridobi segment deljenega pomnilnika (System V shared memory)
```

- signali:

```
kill() // pošlje signal procesu ali skupini procesov
signal() // nastavi obdelovalnik (handler) za določen signal
```

Standardna C knjižnica

- Demo: Napiši program, ki izpiše: ime, dane argumente, in okoljske spremenljivke:
 - environ.c:

```

int main(int argc, char* argv[], char* environ[]) {
    // Izpis imena programa
    printf("Ime programa: '%s'\n", argv[0]);

    // Izpis argumentov
    int i;
    for (i = 1; i < argc; i++)
        printf("%s\n", argv[i]);

    // Izpis okoljskih spremenljivk
    i = 0;
    while (environ[i])
        printf("%s\n", environ[i++]);
}

```

- Demo: Napiši program, ki: izpiše *pid* in *ppid* procesa, izpiše trenutni delovni imenik, ustvari imenik z imenom, ki je podan kot prvi argument, skoči v ustvarjeni imenik, izpiše trenutni delovni imenik.

- demo2.c

```

int main(int argc, char* argv[]) {
    // pid in ppid procesa
    printf("Pid procesa: %d\n", getpid());
    printf("PPid procesa: %d\n", getppid());

    // trenutni delovni imenik procesa
    char cwd[MAXLEN];
    getcwd(cwd, MAXLEN);
    printf("Trenutni imenik procesa: '%s'\n", cwd);

    // ustvarjanje imenika
    char* dir = argv[1];
    if (mkdir(dir, 0777) < 0) {
        fprintf(stderr, "Imenik '%s' ni uspešno ustvarjen.\n", dir);
        exit(1);
    }
    printf("Imenik '%s' uspešno ustvarjen.\n", dir);

    // skok v imenik
    chdir(dir);

    getcwd(cwd, MAXLEN);
    printf("Trenutni imenik procesa: '%s'\n", cwd);

    exit(0);
}

```

Obravnavanje napak

- Funkcije običajno vrnejo status kot njihov rezultat, uspeh: 0, neuspeh: -1.
 - Spremenljivka `errno` nam pa pove kodo zadnje napake ki se je zgodila v sistemskem klicu
 - Obvestilo o napaki lahko izpišemo s funkcijo `perror()`
 - primer:

```

#include <errno.h>
int main(int argc, char** argv){
    if(chdir(argv[1]) < 0){
        int koda = errno;
        perror(argv[0]);
        exit(koda);
    }
    write(1, "Vse je OK.\n", 11);
    exit(0);
}

```

- popravimo kodo, da bo ujela napake, pri našem prejšnjem programu demo2.c:

```
int main(int argc, char* argv[]) {
    // pid in ppid procesa
    printf("Pid procesa: %d\n", getpid());
    printf("PPid procesa: %d\n", getppid());

    // trenutni delovni imenik procesa
    char cwd[MAXLEN];
    getcwd(cwd, MAXLEN);
    printf("Trenutni imenik procesa: '%s'\n", cwd);

    // ustvarjanje imenika
    char* dir = argv[1];
    if (mkdir(dir, 0777) < 0) {
        int koda = errno;
        perror(argv[0]);
        exit(koda);
    }
    printf("Imenik '%s' uspešno ustvarjen.\n", dir);

    // skok v imenik
    if (chdir(dir) < 0) {
        int koda = errno;
        perror(argv[0]);
        exit(koda);
    }

    getcwd(cwd, MAXLEN);
    printf("Trenutni imenik procesa: '%s'\n", cwd);

    exit(0);
}
```

Sledenje izvajanja programa

- Lahko merimo čas izvajanja z ukazom `time`
 - `time ls -R /etc >ls-out.txt` naj bo en tak intenziven proces, ki ga želimo meriti
- Lahko merimo s sistemskim klicem `strace`, statično prevedemo `gcc --static` za bolj pregleden izpis

Uporabniki in datoteke

Uporabniki

- Osnovni ukazi:
 - `whoami` kdo sem?
 - `id` podatki uporabniškega računa
 - `groups` pripadajoče skupine
 - `passwd` nastavitve gesla
 - `$UID` trenutni uid
 - `$HOME` domači imenik
 - `users`, `who`, `w` pregled prijav (oddaljen dostop)
 - `last`, `lastb` zgodovina uspešnih / neuspešnih prijav (oddaljen dostop)
 - `$HOSTNAME` ime računalnika (oddaljen dostop)
- Menjava uporabnika:
 - `sudo` izvajanje podanega ukaza kot drug uporabnik (običajno *superuser*)
 - `su` zamenjava uporabnika (*substitute user*) in zagon lupine
 - `sg`, `newgrp` zamenjava skupine
- Uporabniški računi
 - `useradd` ... ustvarjanje uporabniškega računa
 - `/etc/default/useradd` ... privzete nastavitve

- `/etc/skel/` ... privzete datoteke
- `userdel` ... odstranjevanje uporabniškega računa
- `usermod` ... spreminjanje uporabniškega računa
- `chsh` ... spreminjanje privzete lupine
- `chpasswd` ... spreminjanje gesla
- `chage` ... spreminjanje lastnosti gesla
- `chgrp` ... spreminjanje skupine
- primer:

```
useradd -m -d /home/student/jn1234 -s /bin/bash -g student -c "Janez Novak" jn1234
```

- Uporabniške skupine
 - `groupadd` ustvarjanje skupine
 - `groupdel` brisanje skupine
 - `groupmod` spreminjanje skupine
 - `gpasswd` nastavitve gesla skupine
- Polja v datoteki `/etc/passwd`:
 - `ime:x:uid:gid:polnoIme:domačiImenik:prijavnaLupina`
 - `student:x:1234:1189:Student:/home/student:/bin/bash`
- Polja v datoteki `/etc/shadow`, ki hrani gesla zakodirana s hash funkcijami:
 - `ime:$metoda$sol$koda:ostalo(rok trajanja...)`
 - `student:$6$myVOR2Ad$qa...XAFQdae.:1204201:0123:120`

Tipi datotek

- Branje datotek
 - Datoteke beremo s sistemskimi klici `open()`, `close()`, `write()`, `read()`
 - primer - Program ki izpiše število prebranih bajtov iz datoteke:

```
int main(int argc, char **argv){
    int fd = open(argv[1], O_RDONLY);
    char buf[4096];
    int cnt, count = 0;
    while ((cnt = read(fd, buf, 4096)) > 0){
        write(1, buf, cnt);
        count += cnt;
    }
    close(fd);
    printf("Št. kopiranih bajtov: %d\n", count);
}
```

- primer - Program ki izpiše vsebino imenika:

```
int main(int argc, char** argv) {
    DIR* dir = opendir(argv[1]);
    struct dirent * entry;
    while ( (entry = readdir(dir)) != 0) {
        printf("%s\n", entry->d_name);
    }
    closedir(dir);
}
```

- Trde in mehke povezave
 - `ln -s cilj povezava` ustvarjanje mehke povezave
 - `readlink` izpis cilja mehke povezave
 - `ln cilj povezava` ustvarjanje trde povezave
 - primeri:

```
# simbolična povezava
ln -s /etc/passwd mehka
readlink mehka

# trda povezava
ln /etc/passwd trda
ls -i /etc/passwd trda

# trde povezave in imeniki
mkdir -p 1 2 3
ls -dil "$PWD" . */..
```

Nadzor dostopa

- Lasnik in skupina datoteke
 - `chown` nastavev lastnika
 - `chgrp` nastavev skupine
 - primer

```
# spreminjanje lastnika
chown metka pesem.txt

# spreminjanje skupine
chgrp audio pesem.txt

# oboje hkrati
chown metka:audio pesem.txt
```

- zaščita datotek v Unix sistemih
 - vrste dovoljenj: `r`-read, `w`-write, `x`-execute, `-` prazno
 - sklopi uporabnikov: `u`-user(lastnik), `g`-group, `o`-other, `a`-vsi
- spreminjanje dovoljenj
 - `chmod [augo][+--][rwx-]datoteke`, `+` dodajanje, `-` odvzemanje, `=` nastavljanje
 - primeri:

```
chmod u+w a.txt
chmod go+r b.txt
chmod a-xw c.txt
chmod u=rwx,g=rw,o=r d.txt
```

- `umask` nastavev privzetih dovoljenj

```
umask -S u=rwx, g=rx, o=r
```

- dovoljenji `setuid` in `setgid`
 - primer:

```
int main(int argc, char* argv[]) {
    printf("UID: %d\n", getuid());
    printf("EUID: %d\n", geteuid());
    printf("GID: %d\n", getgid());
    printf("EGID: %d\n", getegid());
}
```

```
gcc uids.c -o uids
chown daemon:games uids ... lastnik in skupina
chmod u+s uids ... setuid
chmod g+s uids ... setgid
```


Procesi

Delo s procesi v lupini

- Pregled procesov
 - `ps` izpis trenutnih procesov
 - `pidof` iskanje pid procesa po imenu
 - `pgrep` napredno iskanje pida procesa po imenu
 - `pstree` izpis hierarhije procesov
 - `top` interaktiven pregled procesov
- Delo s procesi
 - `echo $$` PID lupine
 - `echo $PPID` PPID lupine
 - `echo $BASHPID` PID procesa (lahko gre za podlupino)
 - `sleep seconds` spanje za določen čas
 - `wait` čakanje vseh otrok, da se končajo
 - `wait PID` čakanje da se otrok PID konča
 - `exit status` čakanje izvajajočega se procesa
 - `kill PID` ukinjanje procesa PID
 - primer z podlupinami

```
echo $PPID $$ $BASHPID
(echo $PPID $$ $BASHPID )
( (echo $PPID $$ $BASHPID ) )
```

- demo: [procesinfo.sh](https://github.com/duffy/processinfo.sh)

```
#!/bin/bash
# Preverimo, če je bil podan PID
if [ -z "$1" ]; then
    echo "Uporaba: $0 <PID>"
    exit 1
fi
PID=$1

# Preverimo, če proces sploh obstaja
if [ ! -d "/proc/$PID" ]; then
    echo "Proces s PID $PID ne obstaja."
    exit 1
fi

# Pridobivanje podatkov
EXE=$(readlink -f /proc/$PID/exe 2>/dev/null || echo "Nimaš dovoljenj")
CMDLINE=$(tr '\0' ' ' < /proc/$PID/cmdline 2>/dev/null || echo "Nimaš dovoljenj")
CWD=$(readlink -f /proc/$PID/cwd 2>/dev/null || echo "Nimaš dovoljenj")
ROOT=$(readlink -f /proc/$PID/root 2>/dev/null || echo "Nimaš dovoljenj")
NAME=$(cat /proc/$PID/comm 2>/dev/null)
PPID=$(grep Ppid /proc/$PID/status | awk '{print $2}')
THREADS=$(grep Threads /proc/$PID/status | awk '{print $2}')

# Izpis osnovnih informacij
echo "Izvršljiva datoteka: $EXE"
echo "Ukazna vrstica: $CMDLINE"
echo "Delovni imenik: $CWD"
echo "Korenski imenik: $ROOT"
echo "Ime procesa: $NAME"
echo "PID: $PID"
echo "PPID: $PPID"
echo "Št. niti: $THREADS"

# Izpis odprtih datotek
echo "Odprte datoteke"
if [ -d "/proc/$PID/fd" ]; then
    for fd in /proc/$PID/fd/*; do
        if [ -e "$fd" ]; then
            # Pridobimo številko FD iz poti (zadnji del poti)
            fd_num=$(basename "$fd")
            # Preberemo, kam kaže povezava
            target=$(readlink -f "$fd")
            echo "$fd_num -> $target"
        fi
    done
else
    echo "Nimaš dovoljenj za dostop do seznamov datotek (poskusi s sudo)."
fi
```

Sistemiški klici za delo s procesi

- Ponovitev
 - `fork()` ... ustvaritev novega procesa
 - `exit()` ... končanje tekočega procesa
 - `wait()` ... čakanje na končanje poljubnega otroka
 - `waitpid(...)` ... čakanje na končanje določenega otroka
 - `execXX(...)` ... izvedba programa
- Sistemiški klic `int fork()`
 - shema:

```

int pid = fork();
if (pid > 0) {
    // STARŠ
} else if (pid == 0) {
    // OTROK
} else { // pid < 0
    // NEUSPEH
}

```

◦ primer:

```

int main(int argc, char* argv[]) {
    int pid = fork();
    if (pid < 0)
        perror(argv[0]);
    else if (pid == 0)
        printf("Sem otrok %i s staršem %i.\n", getpid(), getppid());
    else
        printf("Sem starš %i z otrokom %i,\n", getpid(), pid);
}

```

```

starš
|
v
pid=fork() -----> otrok
|               |
v               v
getpid()         getppid()
|               |
v               v
write()          getpid()
|               |
v               v
exit()           write(...)
                  |
                  v
                  exit()

```

- Nastanek sirote

◦ diagram:

```

starš
|
v
pid=fork() -----> otrok
|               |
|               v
v               sleep(60)
exit().....
                | sirota
                v
                .....exit().....

```

◦ program:

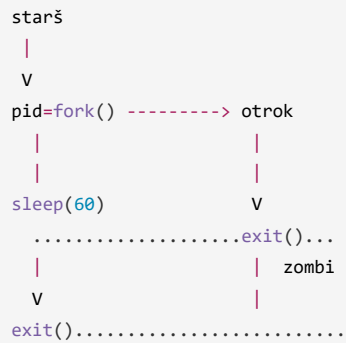
```

int main(int argc, char* argv[]) {
    int pid = fork();
    if (pid < 0)
        perror(argv[0]);
    else if (pid == 0)
        // otrok zaspi za 60 sekund
        sleep(60);
}

```

- Nastanek zombija

- diagram:



- koda:

```

int main(int argc, char* argv[]) {
    int pid = fork();
    if (pid < 0)
        perror(argv[0]);
    else if (pid > 0)
        // starš zaspi za 60 sekund
        sleep(60);
}

```

Unix-podobni sistemi

- Prevzem izhodnega statusa otroka - `wait`

- in čakanje na dokončanje če še ni končan

```

int waitpid(pid, &status, opcije)
int wait(&status)

```

- makro `WIFEXITED(status)` - ali se je program končal z `exit`?

- makro `WEXITSTATUS(status)` - iz spremenljivke status se izlušči izhodni status

```

if(WIFEXITED(status))
    status = WEXITSTATUS(status)

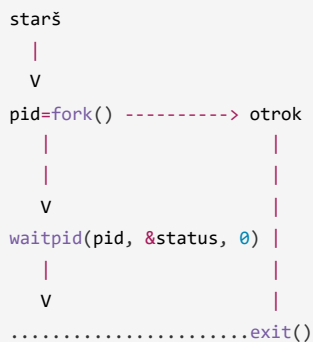
```

- Časovni diagram procesov

- `fork` : stvaritev otroka, otrok dobi vse podatke od starša

- `exit` : oddaja otroka staršu

- `wait` : prejem statusa



- demo program: starš ustvari 3 otroke, izpiše 10 zvezdic, vsako sekundo eno, nato počaka na otroke in prevzame njihove izhodne statuse, izpiše prevzete statuse, hkrati otroci izpisujejo 20 pikic/vejc/podpičji vsako 1 sekundo, nato se končajo s statusom 42 in št otroka.

```

int main(int argc, char** argv){
    pid_t pids[3];
    char znaki[3] = {'.', ',', ';'};

    // ustvarimo 3 otroke
    for(int i = 0; i < 3; i++){
        pids[i] = fork();
        if(pids[i] < 0){
            perror("Fork error!");
        }
        if(pids[i] == 0){
            for(int j = 0; j < 20; j++){
                printf("%c", znaki[i]);
                fflush(stdout);
                sleep(1);
            }
            printf("\nOtrok %d je končal.\n", i + 1);
            fflush(stdout);
            exit(42 + i + 1);
        }
    }

    for(int i = 0; i < 10; i++){
        printf("*");
        fflush(stdout);
        sleep(1);
    }
    fflush(stdout);
    printf("\nStarš je končal z zvezdicami, zdaj čaka na otroke...\n");

    for(int i = 0; i < 3; i++){
        int status;
        pid_t pid = wait(&status);

        if(WIFEXITED(status)){
            printf("Otrok s PID %d se je končal s statusom: %d\n", pid, WEXITSTATUS(status));
        }
    }

    printf("Vsi procesi so zaključeni!\n");
    return 0;
}

```

- Sistemski klic `exec[1v][pPe]`
 - podamo pot do izvršljive datoteke iz ukaza
 - pripone:
 - `l` argumenti ukaza so podani kot argumenti funkcije
 - `v` argumenti ukaza so podani posebej v tabeli
 - `p` uporaba `$PATH` pri iskanju izvršljive datoteke
 - `P` podamo tudi iskalno pot
 - `e` okoljske spremenljivke so podane v tabeli

Procesi v C

- Zagon programa
 - koda:

```

int main(int argc, char* argv[]) {
    int pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(EXIT_FAILURE);
    } else if (pid == 0) {
        execvp(argv[1], &argv[1]);
        perror("exec");
        exit(EXIT_FAILURE);
    } else {
        int status;
        if (waitpid(pid, &status, 0) < 0) {
            perror("waitpid");
            exit(EXIT_FAILURE);
        }
        if (WIFEXITED(status))
            printf("Izhodni status otroka: %i\n",
                WEXITSTATUS(status));
    }
    exit(EXIT_SUCCESS);
}

```

```

starš
|
V
fork() -----> otrok
|               |
|               exec()
V               |
waitpid()        | program
|               |
|               V
.....
|
V
exit()

```

Procesi v Lupini

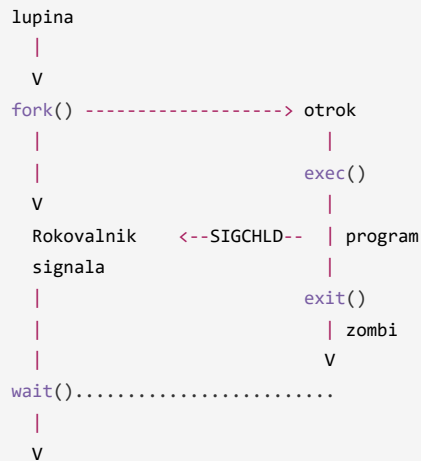
- Zagon programa v ospredju
 - diagram, npr `ls` :

```

lupina
|
V
pid=fork() -----> otrok
|               |
|               exec()
V               |
waitpid(pid, &status, 0) | program
|               |
|               V
.....exit()
|
V
exit()

```

- Zagon programa v ozadju
 - diagram, npr `xeyes &`



- Cevovod
 - npr: `cat /etc/passwd | cut -d: -f7 | sort -u`
 - diagram cevovoda: `prosojnice`
- Procesi v lupini recap
 - Zagon zunanjega ukaza, npr `ls`

```
fork(), exec(), waitpid()
```

- Zagon zunanjega ukaza v ozadju, npr `xeyes &`

```
fork(), exec()
```

- Izvajanje v podlupini: `( vgrajeniUkaz )`, npr `( read line; echo $line )`

```
fork(), waitpid()
```

- Izvajanje v podlupini v ozadju: `( vgrajeni_ukaz ) &`, npr `( echo {24..42} >fri.txt ) &`

```
fork()
```

Signali

Signali

- Signali so programska prekinitev procesa
 - enosmerna komunikacija, kratko sporočilo (številka signala)
- Izvor oz. pošiljatelj signala
 - jedro OS oz poljuben proces
 - jedro uporablja signale za sporočanje izjemnih dogodkov
- Ponor oz. prejemnik signala
 - poljuben proces
 - ciljni proces izvora neposredno ne pozna
- Vrste signalov: signal je podan s številko oz oznako signala (številke si sledijo od 1 naprej, skip 16)
 - `SIGHUP` Hang up
 - `SIGINT` Interrupt (CTRL-C)
 - `SIGQUIT` Quit (CTRL-/)
 - `SIGILL` Illegal instruction
 - `SIGTRAP` Trap (breakpoint)
 - `SIGABRT` Abort (klic abort())
 - `SIGBUS` Bus error(neveljaven naslov)
 - `SIGFPE` FPE error(aritmetična napaka)
 - `SIGKILL` Kill process (brezpogojni)
 - `SIGUSR1` User signal 1
 - `SIGSEGV` Segment violation

- `SIGUSR2` User signal 2
- `SIGPIPE` Broken pipe
- `SIGALRM` Alarm
- `SIGTERM` Terminate process
- `SIGCHLD` Child terminated
- `SIGCONT` Continue process
- `SIGSTOP` Stop process (brezpogojni)
- `SIGTSTP` Stop process from terminal (CTRL-Z)
- Rokovanje signalov
 - Vsak signal ima svoj rokovlanik
 - odziv na signal je kratka rutina, ki se izvede ob prejetju signala
- Način rokovanja
 - Privzeti rokovalnik: `SIG_DFL` - default (za privzeto rokovanje poskrbi OS)
 - Ignoriranje signala: `SIG_IGN` - ignore
 - uporabniški rokovalnik
- Možni privzeti odzivi
 - končanje procesa: proces se zaključi, izhodni status `128 + št.signala`
 - končanje procesa in zapis core datoteke (*core dump*)
 - zaustavitev procesa
 - nadaljevanje procesa
- Uporaba signalov
 - skupine signalov z različnimi nameni
 - obveščanje o sistemskih dogodkih

Vrste signalov

- Ukinjanje procesa
 - privzeti odziv je ukinjanje procesa

```
SIGTERM # generična ukinitvev procesa
SIGINT  # prekinitvev procesa z CTRL+C
SIGQUIT # prekinitvev procesa z CTRL+\, core dump
SIGKILL # brezpogojna ukinitvev procesa
SIGHUP  # sporočilo o končanju pripadajočega terminala
```

- Programske napake
 - privzeti odziv je končanje procesa in zapis `core` datoteke

```
SIGFPE # aritmetična napaka
SIGILL # napačen strojni ukaz
SIGSEGV # kršenje pomnilniške zaščite
SIGBUS  # napačen pomnilniški naslov
SIGABRT # resna napaka oz. klic abort()
```

- Nadzor procesov oz. poslov
 - nadzorni signali:

```
SIGCHLD # sporočilo staršu o končanju otroka
SIGCONT # nadaljevanje zaustavljenega procesa
SIGSTOP # brezpogojna zaustavitev procesa
SIGTSTP # zaustavitev procesa s CTRL-Z
SIGTTIN # posel (proces v ozadju) je bral iz terminala
```

- Razno
 - privzeti odziv je končanje procesa

```
SIGUSR1 # prvi uporabniški signal
SIGUSR2 # drugi uporabniški signal
```

Pošiljanje signala

- Lupina: ukaz `kill`

- pošiljanje signala enemu ali več procesom
- bash ima vgrajeno iz zunanjo različico
- lahko vrne tudi seznam signalov
- signal 0 (obstoj procesa)
- proces -1 (vsi)
- primer:

```
kill -1
kill -1 5
kill -1 TRAP
kill -SIGUSR1 123 456 789
kill -0 $$; echo $?
kill -CONT -1
```

- Sistemski klic `kill(pid, signal)`
 - pošiljanje signala št. *signal* procesu *pid*
- Sistemski klic `raise(signal)`
 - pošiljanje signala samemu sebi

Lovljenje signalov

- Lupina: ukaz `trap`
 - izpis obstoječih (uporabniških) rokovalnikov
 - nastavljanje *pasti* - rokovalnika signala
 - ignoriranje signala
 - ponastavitev rokovalnika
 - primer

```
trap -p
trap "ls" SIGUSR1
trap "echo Zivjo" 2
trap -p SIGINT
trap "" 2
trap - INT
```

- Nov proces
 - otrok deduje rokovalnike starša
 - sistemski klic `fork()` kopira kodo in s tem nudi rokovalnike
- Nov program
 - ima privzete rokovalnike
 - sistemski klic `exec()` ustvari novo sliko procesa iz podanega ukaza, s tem se stari rokovalniki izgubijo
- Uporabniški rokovalnik
 - sistemski klic `signal(signal, sigfun)`
 - ob prejetju signala *signal* se pokliče funkcija *sigfun*
 - podpis rokovalne funkcije
- Rokovalnik končanih otrok

```
void sigchld_handler (int signal) {
    int pid, status, serrno;
    serrno = errno;
    while (1) {
        pid = waitpid(WAIT_ANY, &status, WNOHANG);
        if (pid < 0) {
            perror ("waitpid");
            break;
        }
        if (pid == 0) break;
        notice_termination (pid, status);
    }
    serrno = serrno;
}
```

```
...
signal(SIGCHLD, sigchld_handler);
...
```

Cevi in cevovodi

Preusmerjanje

- Sistemijski klic `dup2(orig, new)`
 - preusmeritev *new* v *orig*
 - če je deskriptor *new* v uporabi, se najprej zapre, nato pa se izvede prevezava
 - orig* lahko nato zapremo
 - shema:

```
orig --> []          orig -> []
                dup2(orig, new) /
new  --> []          new
```

- primer `cat <vhod.txt`:

```
// odpremo datoteko
fd = open("vhod.txt", O_RDONLY);
// izvedemo preusmeritev
dup2(fd, 0);
// zapremo fd, ker ga ne potrebujemo več
close(fd);
// zaženo program
execv("/bin/cat", NULL);
```

- Sistemijski klic `dup(orig)`
 - kot `dup2`, le da za *new* izbere prvo prosto številko deskriptorja
 - navadno tik prej zapremo nek standardni deskriptor
 - shema:

```
orig --> []          orig --> []          orig -> []
                close(new)          dup(orig) /
new  --> []          new  --> []          new
```

Cevi

- Anonimne cevi - primer: starš piše, otrok bere
 - ustvarjanje cevi

```
int fd[2]          starš fd[0] <--- ( cev ) <--- fd[1]
pipe(fd)
```

- razvejitev starša

```
fork()          starš fd[0]          fd[1]
                \          /
                ( cev )
                /          \
otrok fd[0]          fd[1]
```

- priprava

```

starš: close(fd[0])
otrok: close(fd[1])

starš          fd[1]
              /
            ( cev )
            /
otrok fd[0]

```

- branje in pisanje

```

starš: write(fd[1])
otrok: read(fd[0])

```

- sprostitve cevi

```

starš: close(fd[1])
otrok: close(fd[0])

```

Cevovod

- diagram prosojnice
- Primeri

- `ls | grep xyz > bla.txt`

```

void main() {
    int fd[2];
    pipe(fd);

    if(fork() == 0) {
        dup2(fd[1], 1);
        close(fd[0]); close(fd[1]);
        execlp("ls", "ls", NULL);
        perror("exec"); exit(42);
    }
    if(fork() == 0) {
        int f = open("bla.txt", O_WRONLY); //O_CREAT|O_WRONLY|O_TRUNC
        dup2(fd[0], 0);
        dup2(f, 1);
        close(fd[0]); close(fd[1]); close(f);
        execlp("grep", "grep", "xyz", NULL);
        perror("exec"); exit(42);
    }
    close(fd[0]); close(fd[1]); //!!!
    wait(NULL);
    wait(NULL);
}

```

- `ls | cut -d: -f2 | wc -l`

```

void main() {
    int fd1[2];
    pipe(fd1);
    if(!fork()) {
        dup2(fd1[1],1);
        close(fd1[0]); close(fd1[1]);
        execlp("ls", "ls", NULL);
        perror("exec"); exit(42);
    }

    int fd2[2];
    pipe(fd2);
    if(!fork()) {
        dup2(fd1[0], 0);
        dup2(fd2[1], 1);
        close(fd1[0]); close(fd1[1]); close(fd2[0]); close(fd2[1]);
        execlp("cut", "cut", "-d:", "-f2", NULL);
        perror("exec"); exit(42);
    }

    // brez spodnjega closea se cevovod ne bo nikoli prenehal izvajati
    close(fd1[0]); close(fd1[1]);
    //tretji otrok naj deduje samo drugo cev
    if(!fork()) {
        dup2(fd2[0], 0);
        close(fd2[0]); close(fd2[1]);
        execlp("wc", "wc", "-l", NULL);
        perror("exec"); exit(42);
    }
    close(fd2[0]); close(fd2[1]);
    wait(NULL); wait(NULL); wait(NULL);
}

```

Niti-API

Java

- Izvedba niti
 - uporaba vmesnika `Runnable`
 - prekrijemo metode `run`
 - vmesnik:

```

public interface Runnable {
    public void run();
}

```

- Neposredno izvajanje niti
 - razširimo razred `Thread` in povozimo metodo `run()`

```

class MojaNit extends Thread {
    public void run() {
        // opravilo
    }
}

```

- ustvarimo nit: ustvarimo objekt:

```

MojaNit nit1 = new MojaNit();
MojaNit nit2 = new MojaNit("Nitka");

```

- Posredno izvajanje niti
 - ustvarimo razred, ki izvede `Runnable` in povози `run()`

```
class MojaNit implements Runnable {
    public void run() {
        // opravilo
    }
}
```

- ustvarimo nit: razredu `Thread` podtaknemo razred, ki izvede nit

```
Thread nit1 = new Thread(new MojaNit());
Thread nit2 = new Thread(new MojaNit(), "Nitkica");
```

- Zagon in konec niti
 - Zagon: klic `nit.start()`
 - Konec niti: konec metode `run()`, poskrbimo za prizanesljivo končanje (*graceful termination*)

```
volatile private boolean running;
public void run() {
    running = true;
    while (running) {
        // ponavljajoče se opravilo
    }
}
```

- Čakanje niti, da se konča

- klic `nit.join()`
- izjema `InterruptedException`

```
try {
    // kličemo join od niti na
    // kateremo želimo počakati
    nit.join();
} catch (InterruptedException e) {
    e.printStackTrace()
}
```

- Spanje niti

- razred `Thread`

```
static void sleep(long millisecs)
```

- Prepustitev CPE

- razred `Thread`

```
static void yield()
```

Pthreads (Posix Threads)

- Nit implementiramo s funkcijo oblike

```
void* nit (void* arg)
void* // kazalec na karkoli, podatke podamo in vrnemo preko kazalca
```

- Konec niti

- ko se konča funkcija, ki izvede nit ali klic

```
pthread_exit()
```

- Idenititeta niti

- funkcija

```
pthread_t pthread_self()
```

- primeri:

```

void *nit1(void *arg) {
    // id niti
    printf("Sem %li in sem ena uboga nit .\n", pthread_self());
}
void *nit2(void *arg) {
    // prenos niza
    printf("Sem %li in imam %s\n", pthread_self(), (char*)arg);
}
void *nit3(void *arg) {
    // prenos int-a
    printf("Sem %li in imam %li.\n", pthread_self(), (long)arg);
    pthread_exit((void*)42);
}

```

- Stvaritev in zagon niti

- funkcija:

```

pthread_create(
    pthread_t *thread,
    pthread_attr_t *attr,
    void* (*fun)(void*),
    void* arg
)
// primer
pthread_t t1;
pthread_create(&t1, NULL, nit1, NULL);

```

- prenos parametrov: podamo jih, ko kreiramo nit

```

// zagon niti
pthread_t t1, t2, t3;
pthread_create(&t1, NULL, nit1, NULL);
pthread_create(&t2, NULL, nit2, "Juhuhu poletje je tu.");
pthread_create(&t3, NULL, nit3, (void*)42);

```

- lastnosti oz. atributi

```

// priprava atributov
pthread_attr_t attr;
pthread_attr_init(&attr); // privzeti atributi
pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_JOINABLE);
// zagon niti
pthread_t t1;
pthread_create(&t1, &attr, nit1, NULL);
pthread_attr_destroy(&attr);

```

- Čakanje na nit

- nastavitve atributov
 - funkcija `pthread_join()`
 - primer:

```

// priprava atributov
pthread_attr_t attr;
pthread_attr_init(&attr); // privzeti atributi
pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_JOINABLE);
// zagon niti
pthread_t t1;
pthread_create(&t1, &attr, nit1, NULL);
pthread_attr_destroy(&attr);
// čakanje na niti, da se končajo
pthread_join(t1, NULL);

```

- Primer:

```

int main() {
    // priprava atributov
    pthread_attr_t attr;
    pthread_attr_init(&attr); // privzeti atributi
    pthread_attr_setdetachstate(&attr PTHREAD_CREATE_JOINABLE);

    // zagon niti
    pthread_t t1, t2, t3;
    pthread_create(&t1, &attr, nit1, NULL);
    pthread_create(&t2, &attr, nit2, "Juhuhu poletje je tu.");
    pthread_create(&t3, &attr, nit3, (void*)42);
    pthread_attr_destroy(&attr);

    // čakanje na niti, da se končajo
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
}

```

- Ostalo

```

// prepustitev procesorja
pthread_yield()

// prekinitev niti
pthread_cancel()

// preverjanje, če je nit prekinjena
pthread_testcancel()

```

- Primer:

```

void *nit(void *arg) {
    while (1) {
        printf("%ld ", (long)arg);
        pthread_testcancel();
        pthread_yield();
    }
}

int main(int argc, char *argv[]) {
    // ustvarimo 10 niti
    . . .
    // malce počakamo in ustavimo niti
    sleep(3);
    for (i = 0; i < 10; i++){
        pthread_cancel(niti[i]);
    }
    . . .
}

```

PREDAVANJA

Računalniški sistem

Oprema

- Strojna oprema: {Procesor, Pomnilnik, Naprave}
- Programska oprema:
 - Uporabniška PO
 - Sistemska PO: {Jedro OS, Goniliki Naprav, Sistemska orodja, Razvojna orodja, Lupina}

- Ostala PO

Strojna oprema

- Pomniliška hierarhija

Nivo	Dostop	Velikost	Cena/GiB

Registri	0.5 ns	64B	10000
Predpom.	5 ns	16MiB	
DRAM	50 ns	16GB	10
NVMe	10 us	1TiB	0.5
SSD	5 ms	1TiB	0.05
HDD	100 ms	4TiB	

- Obravnava izjem in prekinitev
 - strojne prekinitve: Zahteva V/I naprave ob obravnavi (tipkovnica, miš, mrežna kartica)
 - programske izjeme: izjeme oz. napake pri izvajanju (deljenje z nič, napačen ukaz ...)
 - pasti: posebni strojni ukazi za proženje izjem, sinhrono
 - rokovalnik prekinitev: PSP - prekinitveno servisni program je program ki se izvede točno ob določeni prekinitvi, shranjevanje in obnavljanje stanja procesorja.

```

      program>          program>
    zač servis - |      | - konec servisiranja
               rokovalnik>

```

- Vhodno / Izhodne naprave
 - vrodne naprave (tipkovnica, miš, kamera)
 - izhodne naprave (zaslon, tiskalnik, zvočnik)
 - vmesnik do V/I naprav (pomnilniško preslikan vhod/izhod, ločen vhodno/izhodni prostor)

Programska oprema

- Sistemska PO
 - namenjena upravljanju računalniškega sistema (meja med sistemsko in upo. PO je mehka)
 - paket operacijskega sistema: proizvajalci pogosto dodajajo veliko različne programske opreme, marsikaj ne spada pod SPO (npr minesweeper)
 - to so **operacijski sistem, gonilniki naprav, lupina in sistemski ukazi, virtualizacija in vsebniki**

Sistemska orodja

- Upravljanje datotečnega sistema
 - preverjanje konsistentnosti strukture sistema, čiščenje "odvečnih" datotek, kompresija in kriptiranje diska, analiza praznega prostora, defragmentacija itd.
- Delo z datotekami: file manager, arhiverji (Zip): zgoščanje in razvrščanje
 - varnostne kopije: hramba in obnova
 - sinhronizacija datotek, kriptiranje datotek, urejanje datotek
- Urejevalniki teksta: vim, emacs, Code
- Sistemska orodja in pripomočki:
 - analiza delovanja sistema (testiranje pomnilnika)
 - konfiguracija sistema
 - optimizacija in vzdrževanje računalnika, čiščenje sistema
 - varnost in zaščita
 - mrežna orodja

Lupina

- Lupina je program, ki nudi uporabniku osnovni uporabniški vmesnik, za uporabo, upravljanje in delo z OS.
- Vrste lupin
 - grafične (grafični uporabniški vmesnik, namizno okolje)

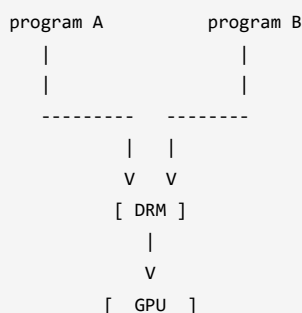
- tekstovne lupine (tekstovni uporabniški umesnik)
 - mešani načini
- Način dela v lupini
 - interaktivna uporaba: uporabnik vnese ukaz in zahteva njegovo izvedbo, lupina izvede ukaz in sporoči status izvedbe
 - neinteraktivna uporaba: minimalna interakcija z uporabnikom, izvajanje zaporedja ukazov, uporabnik zahteva izvedbo skripte

Grafična lupina

- Arhitektura lupine



- Grafični vmesnik
 - namizno okolje (desktop environmet)
 - grafični elementi namenjeni prikazu informacij, okno: ikone, meniji, orodne vrstice ...
 - interaktivni elementi (kurzor itd)
- Prikazni strežnik (display server)
 - komunikacija z aplikacijami po danem **protokolu**
 - posreduje dogodke na vhodnih napravah aplikaciji
 - upravlja z izrisom oken
- Upravitelj oken
 - program, ki nadzoruje postavitev in prikaz oken
 - pogosto je združen s prikaznim strežnikom (Wayland-Linux, Desktop Window Manager-Windows)
 - način upravljanja oken (skladovni, ploščični, kompozitni, dinamični)
- OS - video / GPU podsistem
 - Framebuffer naprava: `/dev/fb0`
 - dostop do video pomnilnika
 - delno upravljanje video naprave
 - branje, pisanje, dostop, preslikan pomnilnik
 - težave ob sočasnem dostopu
- Direct rendering manager
 - naprave `/dev/dri/card0`



Upravitelj oken

- **Skladovni** (stacking) ali lebdeči (floating)
 - omogoča prekrivanje oken po principu sklada, okna so naložena drug na drugega, v ospredju in ozadju
 - izris oken v vrstnem redu glede na oddaljenost: topološka ureditev oken, **slikarski** algoritem in **obratni** slikarski algoritem (od najbližjega do najbolj oddaljenega, ni dovoljeno risanje po že izrisanih oknih)
- Vrste: ploščični, dinamični, kompozitni (Windows)

Ukazna lupina

- Lastnosti ukazne lupine
 - tekstovna lupina
 - uporaba tekstovnega uporabniškega umesnika (command line interface)
 - prosto omogoča napredno uporabo (programiranje, preusmerjanje vhoda in zajem izhoda ukazov, upravljanje poslov)
- Nekaj znanih lupin
 - bash (Bourne again shell) - Linux
 - Z shell (zsh) - macOS
 - Command prompt / Powershell - Windows
- Postopek dela
 - REPL - read evaluate print loop (ponavljanje teh treh faz)
- Vgrajeni in zunanji ukazi
 - ukazi vgrajeni v lupino (shell builtin): `cd, echo, read, if, for, while`
 - zunanji ukazi (external): ukazi kot samostojni progami - `ls, mkdir, cp, head, grep, ps, mount`. Shranjenji so v izvršljivih datotekah nekje v datotečnem sistemu - `/bin/...`
- Izvedba ukaza
 - vgrajeni ukazi imajo prednost pri izvedbi: `echo` ima prednost pred `/usr/bin/echo`
 - iskanje zunanjih ukazov: nastavimo `$PATH` spremenljivko, to je mesto, kjer naj lupina išče ukaz

Skriptiranje

- Skriptiranje
 - Izvorna koda v skriptnem programskem jeziku, ki ga podpira lupina oz. OS
 - Windows: paketna datoteka `batch` za `cmd.exe`
 - Unix: datoteke `.sh`

Ukazna lupina

- Veliko funkcionalnosti
 - avtomatsko dopolnjevanje ukazov in zgodovina
 - preusmerjanje, cevovodi
 - izvajanje v ozadju, upravljanje s posli: `fg, bg, jobs, kill, disown, wait`
 - skriptiranje in pogojno izvajanje: `&& ||, if, fi, for, while, do, done, continue, break...`
 - sklad imenikov: `pushd, popd, dirs`
 - vgrajeni ukazi in programski stavki: `echo, read, cd, pwd, test, type`
- Uporabniški priročnik
 - pomoč za **zunanje** ukaze
 - uporaba: `man ukaz, man man`
 - pomoč za notranje ukaze: `ls --help` ali `help cd`
 - ukaz `type` nam pove kakšnega tipa je ukaz: `type cd` nam izpiše: *cd is a shell builtin*

Terminal

- Standardni vhod, standardni izhod, in izhod za napake
 - `stdin` datotečni tok, deskriptor številka 0
 - `stdout` datotečni tok, deskriptor številka 1
 - `stderr` datotečni tok, deskriptor številka 2
- Oddajen dostop do terminala: `ssh address`

Operacijski sistem

- Kaj je OS?
 - nabor programske opreme (programski sistem)
 - nadzoruje izvajanje programov
 - povezuje uporabnika s strojno opremo
 - deluje kot vmesnik med programi in strojno opremo
- Katere storitve nudi?

- upravljanje z uporabniki
- upravljanje s procesi
- upravljanje s pomnilnikom
- upravljanje z datotečnimi sistemi in datotekami
- Kaj so cilji OS
 - prikladnost uporabe (ease of use)
 - varnost uporabe (security)
 - zanesljivost delovanja (reliability)
 - učinkovitost delovanja (performance)
 - prilagodljivost sistema (flexibility)

Koncepti OS

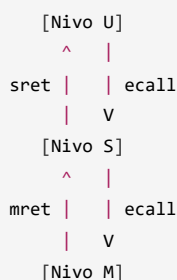
- Abstrakcija
 - posplošitev in skrivanje podrobnosti
 - omogoča poenotenje in združevanje podobnih entitet v eno krovno entiteto
 - primer: abstrakcija datoteke kot hranjene zbirke podatkov
- Virtualizacija
 - mehanizem, ki nekaj ustvari navidezno
 - preslikava navideznega v realno (vmesnik in viri navidezne naprave se preslikajo na vmesnik in vire realne naprave)
 - primer: navidezni procesor, navidezni pomnilnik
- Abstrakcija in virtualizacija sta komplementarna koncepta
 - Primer je navidezni datotečni sistem (VFS), ki nudi enovit dostop do datotek s sistemskimi klici `read()`, `write()`, `open()`, `close()`. Združuje različne naprave in vključuje različne datotečne sisteme.
- Varnost
 - zaupanje v dobro delovanje sistema, dosežemo jo lahko preko mehanizmov zaščite sistema
 - primeri: izolacija procesov, zaščita pomnilka, zaščita V/I naprav, datotek ...
- Sočasnost
 - obstoj več procesov hkrati
 - občutek hkratnega izvajanja procesov, težave in izzivi uporabe sočasnosti
- Persistenca
 - dolgoročni obstoj podatkov in informacij (dnevnik oz. *journaling*)
 - učinkovitost hrambe
 - deljenje informacij (medprocesna komunikacija)

Jedro

- Kaj je jedro OS?
 - to je programska koda, ki vsebuje bistveni del OS
 - vsebuje najpomembnejše storitve OS
 - izvaja se v **priviligiranem** načinu delovanja procesorja.
 - lahko obvladuje celoten sistem: priviligirani strojni ukazi, rokovanje prekinitvev, naslavljanje sistemskih registrov, neposreden dostop do V/I naprav
 - brez tega ni mogoče izgraditi sodobnega OS
- Shema



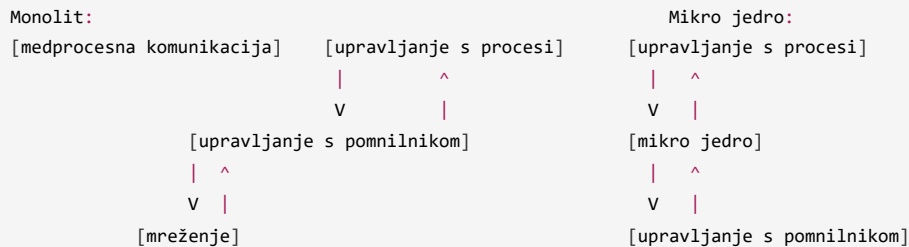
- Procesorski nivoji zaščite
 - **zaščiteni** način: omejena uporaba procesorja, nadzorovan dostop do pomnilnika, neposreden dostop do naprav ni mogoč
 - **priviligirani** način: neomejen dostop do pomnilnika in naprav, nekateri ukazi se lahko izvajajo samo v tem načinu npr. `ecall`, `sret`, `mret`: RISC-V
- Nivoji privilegiranosti
 - x86:
 - 3 : zaščiteni način
 - 0 : privilegirani način
 - -1 : hipervisorski način
 - Arm.
 - PL0 : uporabniški nivo
 - PL1 : jedrni nivo
 - PL2 : hipervizorski nivo
 - Risc-V
 - U : uporabniški nivo (*user mode*)
 - S : supernadzorniški nivo (*supervisor mode*)
 - M : strojni nivo (*machine mode*)



- Komunikacija med jedrom in strojno napravo
 - **naprava**: dejanska naprava, ki omogoča neko opravilo
 - **kontrolniki** naprav: elektronska vezja, ki razumejo ukaze in jih posredujejo napravi
 - **vmesnik strojne opreme**: mehanizem programskega podajanja ukazov napravam: npr pomnilniško preslikan V/I
 - **gonilniki** naprav (device drivers): programska koda, ki zna upravljati z neko napravo, navadno so gonilniki del jedra

Arhitektura jedra

- To je **struktura** in **način povezovanja** med posameznimi deli jedra
 - Vrste jedrnih arhitektur: `monolitno jedro`, `monolitno modularno jedro`, `mikro jedro`, `hibridno jedro`
- Monolitno jedro
 - Je velik kos strojne kode, ki teče v jedrnem nivoju v skupnem naslovnem prostoru. Vsebuje celoten OS (podsistemi, gonilniki)
 - Hitrost: posamezni deli OS lahko med sabo hitro komunicirajo preko navadnih klicev funkcij
 - Slabosti: napaka v enem delu jedra lahko pomeni sesutje celotnega OS, programska koda je težje obvladljiva. Sprememba izvorne kode pomeni ponovno prevajanje celotnega jedra.
- Monolitno modularno jedro
 - navkljub modularnosti omogoča modularno zasnovo jedra: modul navadno vsebuje gonilnik naprave
 - dinamičnost: module je moč vložiti (*insert*) in tudi izločiti (*remove*) iz jedra tekom samega izvajanja sistema (ni potreben reset)
 - primeri: BSD, Linux, OS-9
- Mikro jedro (μ -jedro)
 - vsebuje samo osnovne funkcionalnosti (proces, pomnilnik, medprocesna komunikacija)
 - ostale funkcionalnosti so izvedene preko procesov v uporabniškem načinu (uporaba medprocesne kombinacije)
 - prilagodljivost, porazdeljenost, varnost, enostavnejša implementacija
 - primeri so: Mach, L4, seL4, Minix 2
- Monolitno vs mikro jedro



- Hibridno jedro
 - zasnova je mikro jedrna (ločeni servisi)
 - izvedba je monolitna (skupen naslovni prostor)
 - primer: Windows NT, iOS, macOS
- Ostala jedra
 - nano jedro, piko jedro ...
 - exokernel
 - unikernel (namensko jedro za izbrano aplikacijo)

Sistemske klice

Sistemske klice

- Kaj je sistemski klic?
 - To je mehanizem preko katerega **uporabniški program** zahteva **jedro storitev**.
 - Klic podprograma v jedru, ki implementira zahtevano storitev (podobno navadnemu klicu funkcije)
 - Vsak sistemski klic ima svojo **številko**: npr 1 .. sistemski klic `exit()` . Prejme lahko tudi argumente `exit(32)` . Prenos številke in argumentov poteka preko registrov in sklada.
- Tabela rokovalnikov sistemskih klicev
 - i-ti element v tabeli je **naslov** rokovalnika
 - vrstni red je fiksni že dolgo časa in se ne spreminja
 - številka sistema klica je enaka indeksu v tabeli rokovalnika
 - primer:

```

1 .. exit()
2 .. fork()
3 .. read()
4 .. write()
itd.

```

- Preklop nivoja zaščite procesorja
 - **jedro** ki nudi storitev, teče v privilegiranem načinu
 - **kličoči program** ki zahteva storitev teče v zaščitenem načinu. Direktni klic podprograma v jedru bi sprožil izjemo.
 - potrebna je pomoč **strojne** opreme za izvedbo preklopa v privilegiran način
- Sistemski vmesnik - preklop v jedro
 - **namenski strojni ukaz** za sistemski klic in tudi za vrnitev iz njega, procesor naredi preklop in pokliče nameščeni rokovalnik sistemskih klicev v jedru
 - primeri:
 - x86 `sysenter/sysexit`
 - x86 AMD `syscall/sysret`
 - ARM `svc/restore PC from LR`
 - RISC-V `ecall/sret(mret)`
 - **programska prekinitve**: lahko jo sprožimo le programsko.
 - Procesor izvede preklop v jedrni način.
 - Rojovalnik prekinitve je podprogram v jedru, ki skrbi za obdelavo prekinitve.
 - Procesor naredi preklop in pokliče nameščeni rokovalnik prekinitve v jedru.
 - Vsak OS ima svojo izbrano številko prekinitve. npr: x86: `INT/IRET`, Windows: `0x2E`, Linux: `INT 0x80`
- Ostali mehanizmi preklopa
 - **klicna vrata** (call gate)

- x86 specifično, zahteva oddaljeni klic v drug segment
- zahtevna inicializacija segmentov
- možno preklapljanje tudi v manj privilegirani nivo
- **pomnilniška vrsta**
 - uporabno za veliko število klicev
 - sistemske klice postavljamo v vrsto
 - jedro periodično pregleduje vrsto
- Primer preklopa: RISC-V
 - izvedba klica: namenski ukaz `ecall`
 - št.klica: register `a7`
 - argumenti klica: registri `a0 .. a6`
 - vračanje iz klica: namenski ukaz `sret`
- Primer preklopa: x86 in Linux
 - programska prekinitev: strojni ukaz `int 0x80`
 - št klica: register `eax`
 - argumenti klica: registri `ebx, ecx, edx, esi, edi, ebp`
 - vračanje iz klica: strojni ukaz `iret`
- Izvedba sistemskega klica
 - priprava na sistemski klic: podajanja št. sistemskega klica in argumentov
 - vstop v jedro: preko sistemskega vmesnika, prekop v privilegiran način, proženje rokovalnika
 - izvedba rokovalnika sistemskega vmesnika: preverjanje št. klica in klic specifičnega rokovalika
 - izvedba rokovalnika sistemskega klica: navaden klic rutine znotraj jedra
 - izstop iz jedra: prekop nazaj v uporabniški način
- Primerjava: sistemski klic in klic funkcije
 - sistemski klic je počasnejši, saj zahteva prekop nivoja zaščite
 - izvedba samega rokovalika klica je zahtevnejša (preverjanje argumentov, uporaba tabele handlerjev, klic rokovalnika)
 - podpora procesorja: običajen funkcijski klic `call, jmp, jal`, poseben mehanizem za sistemski klic: ukaz: `ecall, sysenter, syscall, svc`
- OS kot programska knjižnica
 - Če bi uporabljali le klice funkcij za izvedbo sistemskih klicev bi bila vprašljiva varnost programov. Lahko bi prišlo do lukenj v takem sistemskem klicu, kar lahko sesuje celoten OS, lahko omogoči vdor v jedro in posledično vdor v katerikoli program.
- Varnost sistemskih klicev
 - Sistemski klic je tudi mehanizem zaščite za nadzor operacij, ki jih izvajajo procesi. Ne more vsakdo delati vsega. Poteka avtorizacija uporabnikov/procesov/opravil.
 - Primer preverjanja: preverjanje št. klica, preverjanje podanih argumentov klica, preverjanje ali ima proces dovoljenje za dostop do zahtevanih virov. Preverjanje ali ima proces dovoljenje za izvedbo zahtevane operacije.

Ovojne funkcije sistemskih klicev

- Zahtevnost izvedbe sistemskega klica
 - neposredna izvedba je težavna, saj zahteva programiranje v zbirniku, rokovanje z registri in vstop v jedro.
- Ovojne funkcije (*wrapper function*)
 - njen namen je klic druge funkcije oz. izvedba sistemskega klica (preverjanje argumentov, in vstop v jedro)
 - Std C knjižnica: `stdlib.h, unistd.h, stdio.h,`
 - npr klic `fork()` v uporabniškem prostoru, ki je sistemski klic števika `2`, in rokovalnik sistemskega klica v jedrnem prostoru `sys_fork()`
- Operacijski sistem Windows NT
 - dinamična knjižnica `ntdll.dll` (Native API), se sicer ne uporablja pogosto
 - dinamična knjižnica `kernel32.dll` (Windows API): ovojne funkcije ki kličejo tiste iz Native API-ja. To je osnovni API operacijskega sistema Windows.
 - create process: knjižnica `processthreadsapi.h`, ovojna funkcija `CreateProcessA()`
- Izveba sistemskega klica
 - **neposredno**: nastavitve registrov in vstop v jedro v zbirniku
 - preko **specifične ovojne funkcije**
 - preko **splošne ovojne funkcije**: `syscall()`
 - **posredno** preko ostalih funkcij: npr `printf()` kliče `write()`

Standardi

- Aplikacijski programski vmesnik

- **API**: application programming interface
- vmesnik za uporabo programskih knjižnic
- temelji na simbolični predstavitvi (npr imena funkcij, razredov ipd.)
- Aplikacijski dvojiški vmesnik
 - **ABI**: application binary interface
 - temelji na številski predstavitvi (številska oznaka funkcij, prenos argumentov v registrih)
- POSIX - standard IEEE 1003 (Portable Operating System Interface)
 - prenosljivi vmesnik operacijskega sistema: definira programski vmesnik med aplikacijami in OS, predpisuje funkcije, ukazno lupino, okoljske spremenljivke, orodja lupine, datotečno hierarhijo, knjižnico za nitenje,
 - vzrok standardizacije
 - obstaja veliko različnih UNIXov
 - standard omogoča prenosljivost programov
- SUS standard (Single unix Specification)
 - certificirani Unix sistemi (macOS)
 - ne-certificirani Unixu podobni sistemi (Linux)
 - ostali ne-certificirani sistemi (WSL)

Uporabniki

Uporabniški račun

- Uporabnik
 - oseba, ki uporablja računalniški sistem oz. ima ustvarjen uporabniški račun
- Število uporabnikov OS-a
 - **brezuporabniški** ali enouporabniški OS: ni razlikovanja med uporabniki, ni nadzora nad porabo virov
 - **večuporabniški** OS: razlikovanje in razpoznavanje uporabnikov, sočasna uporaba sistema s strani večih uporabnikov, lastništvo virov, nadzor porabe virov
- Uporabniški račun
 - mehanizem ki omogoča razlikovanje med uporabniki, prijavo v sistem, ločevanje virov
 - to je skupek podatkov o uporabniku: osebni podatki, podatki za prijavo v sistem (user_id, geslo, domači imenik, prijavna lupina), podatki za mehanizem zaščite (uporabnikove skupine)
- Vrste uporabniških računov:
 - običajen uporabnik
 - administrator (vzdrževalec sistema, ima več ali vse pravice)
 - superuser (uporabnik, ki začasno dvigne svoje pravice)
 - gost (anonimni običajni uporabnik, njegove nastavitve se ne hranijo)

Prijava v sistem

- Prijava v sistem
 - **identifikacija** (uporabniško ime)
 - **overitev / avtentikacija** (preverjanje istovetnosti danih podatkov, potrjevanje identitete)
 - Overitveni dejavniki
 - dejavniki znanja (nekaj kar uporabnik ve): gesla, osebni podatki, geste, podpis ...
 - lastniški dejavniki (posedovanje **fizičnih** naprav): ključ, osebna, ...
 - prirojeni dejavniki (prstni odtis, zenica, obraz ...)
 - Overitev
 - **enodejavniška** overitev (single-factor-authentication)
 - **večdejavnniška** overitev (multi-factor-authentication)
 - Način dostopa do računalnika
 - **lokalna** prijava
 - **oddaljena** prijava: najprej se prijavimo v lokalni računalnik, prijava v oddaljeni računalnik preko lokalnega, dostop preko omrežja
- ssh oz secure shell

Uporabniki v Linux

- Administrator

- `uid = 0`, `gid = 0`
 - uporabniško ime `root`
 - domači imenik `/root`
 - ima vsa dovoljenja in odgovornost
 - v sodobnih distribucijah je običajno onemogočen
- Superuporabnik (sudoer)
 - uporabnik ki začasno dvigne svoje pravice z ukazom `sudo`
- Programski uporabniki
 - uporabniški računi za programe, `uid > 0`, običajno med 200 in 999
- Ostali uporabniki
 - skoraj poljubno uporabniško ime
 - običajno `uid >= 1000`
 - domači imenik se nahaja v `/home/<username>`
- Uporabniški računi: datoteki `/etc/passwd` in `/etc/shadow`
- Soljenje gesel
 - naključen niz dodan geslu pred zgoščevanjem
 - postopek soljenja je dokaj zapleten
 - namen: enaki gesli + različna sol imata različno zgoščeno vrednost. Onemogočanje napada z mavrično tabelo.
- Uporabniške skupine
 - datoteka `/etc/group`
 - hrani sezname uporabnikov, ki še pripadajo skupini poleg skupine zapisane v `/etc/passwd`

`imeSkupine:x:gid:seznamUporabnikov`
- datoteka `/etc/gshadow`
 - gesla skupin

Datoteke

Logična organizacija podatkov

- Datoteka:
 - Je osnovna zaključena zbirka podatkov. Dolgotrajna obstojnost (odvisna od pomnilnega medija), jo lahko naslovimo oz. identificiramo preko imena.
 - vsebina datoteke: podatki ki jih hrani
 - atributi (meta podatki): pripadajoči podatki o datoteki
- Imenik
 - omogoča združevanje (sorodnih datotek)
 - vsebuje datoteke in druge imenike oz. vsebuje seznam datotek in pripadajočih atributov
 - vsebino imenika OS razume, vsebina je seznam datotek
- Hierarhična struktura imenikov
 - gnezdenje imenikov in datotek (poljubna globina gnezdenja)
 - odnosi med imeniki: **podimenik** imenik vsebovan v danem imeniku, **nadimenik** imenik, ki vsebuje dani imenik
 - **korenski** imenik: imenik, ki nima nadimenika (ni nikjer vsebovan)

Datoteke

- Naslavljanje datotek
 - **pot** do datoteke: seznam imen imenikov od ustreznega izvirnega imenika do dane datoteke
 - **absolutna pot**: pot ki se prične v *korenskem imeniku*
 - **relativna pot**: pot ki se prične v *trenutnem delovnem imeniku* - `pwd`

```

/ korenski imenik, ločilo v poti (Unix)
\ korenski imenik, ločilo v poti (Windows)
. trenutni imenik
.. imenik starša trenutnega imenika
~ domači imenik uporabnika

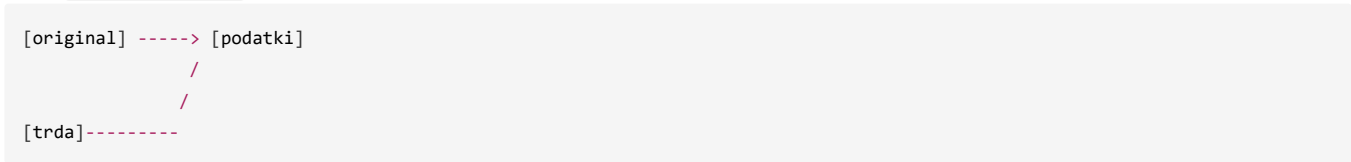
```

- Abstrakcija datoteke

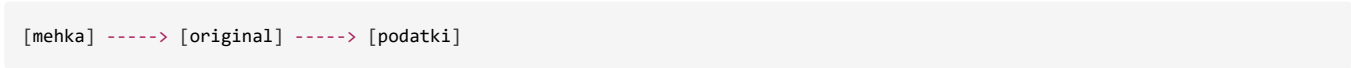
- datoteka je pogosto splošnejši koncept
- različni **tipi datotek**
- OS nekatere tipe posebej obravnava

oznaka	opis	primer
-	navadna datoteka	/etc/passwd
d	imenik (directory)	/bin
l	simbolična povezava (link)	/usr/src/linux
b	bločna naprava	/dev/sda
c	znakovna naprava	/dev/tty
p	imenovana cev (named pipe)	
s	lokalna vtičnica (local socket)	

- Navadna datoteka
 - vsebina datoteke: poljubna interna struktura oz. format, omogoča notranje operacije (pisanje, zapiranje, branje, pisanje)
 - vrsta (navadne) datoteke: različne vrste datotek glede na vsebino, pogosto del imena (npr. končnica) podaja vrsto, veliko različnih vrst...
- Imenik
 - To je datoteka posebnega tipa, njena vsebina je **seznam imeniških vnosov**, kjer vsak imeniški vnos ustreza neki datoteki
 - **imeniški vnos** (directory entry): **ime** datoteke, **kazalci** na ostale podatke (vsebina, meta-podatki), znotraj imenika ni mogoče imeti imeniških vnosov z enakim imenom
- Trda povezava (*hard-link*)
 - to ni nov tip datoteke
 - to je **dodaten imeniški vnos** (za isto datoteko)
 - torej je trda povezava enakovredna imeniškemu vnosu
 - ukaz: `ln original trda`



- Simbolična (mehka) povezava (*soft-link*)
 - to je datoteka posebnega tipa
 - vsebina datoteke (simbolične povezave) je **naslov** ciljne datoteke (neka pot do nje)
 - večina ukazov avtomatsko sledi tej povezavi da operira nad ciljno datoteko
 - ukaz: `ln -s original mehka`



- Bločna in znakovna naprava
 - dva posebna tipa datotek
 - branje in pisanje takšne datoteke se neposredno nanaša na napravo
 - **bločna** naprava (dostop po blokih npr. 4096B)
 - **znakovna** naprava (dostop po znakih oz bajtih)
- Cev (*pipe*) in vtičnica (*socket*)
 - mehanizma za medprocesno komunikacijo
 - naslavljanje preko imen datotek (nadzor dostopa preko datotečnih dovoljenj)
 - **imenovana cev** (named pipe), FIFO.
 - ustvarimo z `mkfifo`
 - nudijo enak vmesnik kot datoteke
 - **lokalna vtičnica** (local socket), *Unix domain socket*
 - poseben vmesnik, kot neke internetne vtičnice

Kodiranje datotek

- ASCII: 7bitno kodiranje, 95 vidnih in 33 nevidnih kontrolnih kod
 - razširitev ASCII 8bitov
- Standard Unicode
 - UCS - universal characters set
 - svetoven repertoar znakov, emoji 🤪

- 21 - bitni kodni prostor (codepace)
- prvih 256 znakov je enakovrednih ISO-IEC 8859-1
- Različna kodiranja
 - UTF unicode transformation format
 - UTF-8: Unix, spet
 - UTF-16: Windows, ena ali dve dvobajtni besedi
 - UTF-32: 4 bajti, prostorsko neučinkovito
- Standard UTF-8
 - Razširjen ASCII, se ujema v prvih 128 znakih
 - porabi 1-4 bajtov na znak
- Skok v novo vrstico
 - **CR** skok na začetek vrstice (Carrige Return)
 - **LF** naslednja vrstica (Line Feed)
 - CR-LF Windows, LF - Unix

Nadzor dostopa

Varnost

- Vrste varnosti
 - Informacijska varnost
 - Kibernetska varnost
 - računalniška varnost, IT varnost, digitalna varnost
 - Omrežna, internetna in spletna varnost
 - Oblačna, aplikacijska in programska varnost
 - Varnost infrastrukture, podatkovnih baz, računalniških sistemov, operacijskih sistemov
- Varovanje **občutljivih informacij**: **tajnost** (secrecy), **zasebnost** (privacy)
- Ranljivost sistema (*system vulnerability*)
 - sistem je izpostavljen možnosti neželene uporabe
- Varnostna pretnja (*security threat*)
 - potencialna negativna aktivnost, katere posledica je lahko neželjeno delovanje sistema
- Varstvo (*safety*)
 - zagotavljanje želenega delovanja sistema
 - varovanje pred škodo zaradi nenamernih negativnih aktivnosti
 - funkcionalna varnost, pravilnost
- Varnost (*security*)
 - izogibanje neželenemu delovanju sistema
 - varovanje pred škodo zaradi namernih negativnih aktivnosti
 - kriminal, vdiranje v sisteme
- Cilji varovanja
 - preprečiti oz. zmanjšati verjetnost nepooblaščenega oz. neprimerne dostopa do podatkov in sistemov
 - vzpostaviti oz. ohraniti zaupanje v želeno delovanje sistema
- Vloga OS
 - ponuditi mehanizme za zaščito (protection) oz. varovanje
 - pravila uporabe oz. **varnostna politika** (policy)
 - OS vzpostavlja **nadzor dostopa do virov**

Občutljivost podatkov

- Triada CIA:
 - **Zaupnost** (*confidentiality*) - zaščita pred nepooblaščenim branjem
 - **Celovitost** (*integirty*) - zaščita pred nepooblaščenim pisanjem
 - **Razpoložljivost** (*integirty*) - preprečevanje DDoS napadov
- Pristnost, avtentičnost (*authenticity*) - dokaz izvora sporočila za prejemnik
- Načela načrtovanja varnih sistemov
 - ekonomičnost mehanizma - naj bo preprost in nezapleten, vgrajen v najnižji mogoči sloj sistema

- odprta zasnova (*open source*) - mehanizem naj bo javen, predpostavi, da napadalec pozna vse podrobnosti, a mehanizem kljub temu omogoča varovanje
- varne privzete nastavitve, privzeto dovoljenje naj bo "ni dostopa", legitimni dostop zavržen lažje odkrijemo, uporabnik se pritoži administratorju, neavtoriziran dostop odobren težje odkrijemo
- popolno in sprotno preverjanje: aktualna dovoljenja naj se preverijo ob dejanju, sprememba dovoljenj tako ne povzroči težav
- najmanjši privilegiji: uporabniki naj imajo najmanjše še možne privilegije, za izvedbo zelenih dejanj
- ločevanje privilegijev: kritična dejanja naj zahtevajo potrditev s strani več uporabnikov
- najmanjši skupni mehanizem za različne uporabnike ali proces uporabi ločene podatkovne strukture ali mehanizme
- shema varnosti naj bo uporabniško prijazna: prezapleteni mehanizmi se ne bodo uporabljali

Nadzor dostopa

- Nadzor dostopa do datoteke:
 - določa kdo lahko kaj počne s čim
 - **kdo?** subjekt
 - uporabnik, skupina, proces, območja zaščite
 - npr. lastnik datoteke, skupina datoteke, ostali
 - **kaj?** dovoljenje
 - dovoljenja za operacije nad datoteko
 - npr. nič, preverjanje obstoja, izvajanje, branje, dodajanje, pisanje, spreminjanje zaščite, brisanje
 - **s čim?** objekt, vir
 - objekti zaščite so navadno datoteke, vendar lahko tudi naprave, pomnilnik itd.
- Matrika nadzora dostopa
 - stolpci: objekti nadzora dostopa
 - vrstice: subjekti, ki dostopajo
 - elementi: način dostopa (npr. nabor dovoljenj)

	Datoteka 1	Datoteka 2
Uporabnik A	lastnik: R, W	R, X
Uporabnik B		R
Uporabnik C	W	

- Nadzorni sistem dostopa (access)
 - vsak vir ima svojo seznam dostopa (access list)
 - seznam vsebuje dekompozirano matriko po stolpcih
- Seznam zmožnosti (capabilities list)
 - vsak subjekt ima svoj seznam zmožnosti
 - zmožnost podaja dovoljenja uporabe virov
 - seznam vsebuje dekompozirano matriko po vrsticah

Zaščita datotek

- Osnovna zaščita dovoljenj
 - vrste dovoljenj: `r-read`, `w-write`, `x-execute`
 - sklopi uporabnikov: `u-user`, `g-group`, `o-other`, `a-all`

```
u  g  o
|  |  |
v  v  v
rwxrw-r--
```

- pomen dovoljenj glede na tip datoteke

zank	datoteka	imenik
r	branje datoteke	izpis imenika, npr ukaz ls
w	pisanje oz spreminjanje datoteke	spreminjanje imenik (ustvarjanje in bris im vnos.)
x	izvajanje datoteke	vstop v imenik, npr ukaz cd.

- Kdo lahko pobriše datoteko?

- o običajna zmota: njen lastnik
- o brisanje datoteke = brisanje imeniškega vnosa
- o torej: **kdor ima dovoljenje pisanja v imenik, kjer je datoteka**
- o podobno velja za ustvarjanje novih datotek

```
drwxrwxr-x zala devops ./
drwxr-x--- sef boss ../
-rwxr-x--- vid tester brisi.me
```

Datoteko brisi.me lahko pobriše le Zala, ker je lastnik trenutne mape.
In vsi tisti, ki so v grupi [devops] imajo "w" (drugo polje gledamo rwx)
Ostali pa nimajo "w", gledamo polje other.

- Omejeno brisanje (*restricted deletion*)

- o bit oz. oznaka **t** pomeni omejeno brisanje, sprememba privzetega delovanja, datoteko v imeniku lahko odstrani lastnik datoteke
- o uporaba: imenik `/tmp` za začasne datoteke

```

|
v
drwxrwxrwt root root tmp
```

- Zagon programa

- o nastali proces dobi **dovoljenja trenutnega uporabnika** in skupin,
- o bit `setuid`, oznaka `u+s`
 - zagnani program dobi dovoljenja lastika
- o bit `setgid`, oznaka `g+s`
 - zagnani program dobi dovoljenja skupine

```
getuid() ... UID lastinka procesa
getgid() ... GID skupine, kateri pripada proces
geteuid() ... aktualni [effective] UID lastinka procesa
getegid() ... aktualni [effective] GID skupine, kateri pripada proces
```

Upravljanje s pomnilnikom

Naslovni prostor

- Pomnilnik
 - o fizični pomnilnik: je pomnilnik, ki je dejansko povezan s procesorjem, proces hrani informacije o fizičnem pomnilniku
 - o naslovni prostor: pomnilnik ga vidi kot proces, OS abstrahira fizični pomnilnik zaradi lažje uporabe
- Zgodnji preprosti sistemi (ZX Spectrum 48K)
 - o brez posebnega mehanizma, logični naslov = fizični naslov
 - o delitev pomnilnika: en del je bil rezerviran za OS, ostalo za proces (trenutni program)
- Večprogramiranje
 - o več procesov v pomnilniku
 - o deljenje pomnilnika, preklapljanje procesov, zaščita procesov
 - o izgled fizična pomnilnika

```

0kb                                     1MB
-----
| OS | Proces A | Proces B | Proces C |
-----
```

- Naslovni prostor

- o pomnilnik ga vidi kot proces, vidi samo svoj kos
- o sestavljen je iz: kode, kopica, sklad, drugo
- o preslikava virtualnega/logičnega naslova v fizični naslov
- o shema naslovnega prostora:

0kb 128kB

| koda | kopica | -> prosto <- | sklad |

- Naslovni prostor programov v C je zelo podoben

- `koda` strojna koda (text)
- `init data` inicializirani podatki (data)
- `uninit data` neinicializirani podatki (bss)
- `kopica` dinamična alokacija (heap)
- `sklad` (stack)
- `args` argumenti ukazne vrstice
- `env` okoljske spremenljivke

0kb 128kB

| koda | init data | uninit data | kopica | -> prosto <- | sklad | args + env |

- Določanje naslovov
 - čas prevajanja (compile time): zbirnik ali prevajalnik določi naslove simbolov oz. spremenljivk
 - čas povezovanja (link time): zlaganje več prevedenih modulov skupaj
 - čas nalaganja (load time): prenaslovljiva koda (relocatable code), nalaganje na poljubno lokacijo v pomnilniku
 - čas izvajanja (run time): premikanje in zamenjava procesov (zunanji pomnilnik)
- Zaščita procesa
 - **izolacija** naslovnega procesa: os poskrbi za ustreznost preslikave VA (virtualni naslov) -> PA (fizični naslov)
 - **ščitenje** delov naslovnega prostora: preverjanje naslovov, dovoljenja za operacije (pisanje, branje, izvajanje), past zaščite

Pomnilniški API - libc

- Alokacija pomnilnika:
 - sklad
 - implicitna alokacija in dealokacija preko prevajalnika
 - avtomatski pomnilnik
 - lokalne spremenljivke, argumenti funkcij, izvajalni podatki
 - kopica
 - explicitna alokacija pomnilnika

```
void* malloc(size_t size)
calloc(...) in realloc(...)
```

- dealokacija: explicitna dealokacija ali garbage collection (avtomatsko)

```
free(void* ptr)
```

Pomnilniške storitve OS

- Kaj naredi `exit()` glede pomnilnika?
 - sprosti naslovni prostor procesa
 - puščanje pomnilnika je možno le znotraj procesa, vendar to ni huda težava, lahko pa povzroči sesutje procesa ...
- Velikost podatkovnega segmenta
 - oz. velikost kopice lahko nastavljamo takole:

```
// Nastavljanje velikosti
void* brk(void* addr)

// Spreminjanje velikosti
void* sbrk(int incr)

// Trenutna velikost
sbrk(0)
```

- Pomnilniško preslikana datoteka
 - os preslika datoteko v nek del prostega prostora
 - sistemska klica `mmap()` in `munmap()`

```
// Alokacija in preslikava
void* mmap(
    void* addr, size_t len, // blok v naslovnem prostoru
    int prot, int flags,    // zaščita (RWX) in ostalo
    int fd, off_t offset    // del datoteke
)

// Sproščanje pridobljenega pomnilnika
void* munmap(
    void* addr, size_t len
)
```

- primer: anonimna nepersistentna preslikava

```
char *p = mmap(
    NULL, 4096,
    PROT_READ | PROT_WRITE,
    MAP_PRIVATE | MAP_ANONYMOUS,
    -1, 0);
...
munmap(p, 4096);
```

- primer: nepersistentna preslikava datoteke

```
int fd = open("~/mybigfile", O_RDONLY);

int size = 4096; // use fstat() for file len

char *p = mmap(
    NULL, size,
    PROT_READ,
    MAP_PRIVATE,
    fd, 0);

write(1, p, size);

munmap(p, size);

close(fd);
```

- primer: persistentna preslikava datoteke

```
int fd = open("~/mybigfile", O_RDWR);

int size = 4096; // use fstat() for file len

char *p = mmap(
    NULL, size,
    PROT_READ, | PROT_WRITE
    MAP_SHARED, // <----- to glej
    fd, 0);

p[0] = 42;

munmap(p, size);

close(fd);
```

- primer: souporaba pomnilnika z otroci

```

int size = 4096; // use fstat() for file len

char *p = mmap(
    NULL, size,
    PROT_READ | PROT_WRITE,
    MAP_ANONYMOUS | MAP_SHARED,
    -1, 0);

if (fork() == 0) {
    // write something to p
} else {
    wait(NULL);
    // read from p
    munmap(p, size);
}

```

- Randomizacija naslovnega prostora
 - ASLR - address space layout randomization
 - težje za napadalce da najdejo okvarjeno kodo, da skočijo vanjo, če so ključni deli naslovnega prostora naključno razporejeni.

Virtualizacija pomnilnika

Virtualizacija pomnilnika

- Cilji
 - proces vidi sklenjen kos pomnilnika
 - izolacija procesov
 - procesi ne morejo vplivati drug na drugega
 - tudi OS je izoliran od procesov
 - uporaba mehanizma zaščite
 - transparentnost
 - proces se ne zaveda virtualizacije
 - učinkovitost
 - preslikava naslovov mora biti hitra (strojna podpora)
- Preslikava naslovov
 - podpora strojne opreme
 - OS skribi za konfiguracijo
 - MMU izvaja preslikovanje
 - TLB skrbi za učinkovitost preslikovanja

```

-----
navidezni | PA = f(VA) \ fizični naslov
naslov    |          /
-----

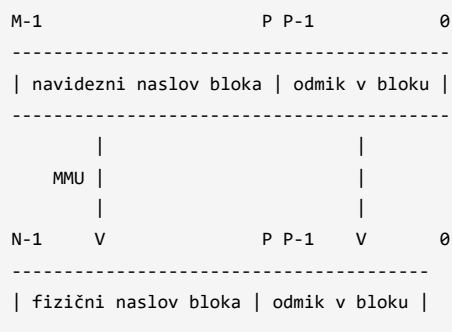
```

- izvedbe preslikovanja
 - celotna preslikovalna tabela (izvedba za vse navidezne naslove ni mogoča)
 - premeščanje (relocation)
 - naslovni prostor premestimo na poljubno mesto v fizičnem pomnilniku
 - segmentacija
 - razdelitev pomnilnika glede na bloke različnih velikosti
 - ostranjevanje (paging)
 - razdelitev pomnilnika na bloke enakih velikosti
 - segmentacija in ostranjevanje
 - segmenti so razdeljeni na strani
- Statično premeščanje
 - program se prevede na naslov 0
 - nalaganje programa

- program se naloži na izbrani ciljni fizični naslov
- težavna zaščita:
 - statični naslovi OK, dinamični naslovi TEŽAVA
- težavno ponovno premeščanje programov
- statično premeščanje se uporablja le kadar ni strojne podpore za virtualizacijo pomnilnika
- Dinamično premeščanje
 - nalaganje programa
 - program se naloži na izbrani ciljni fizični naslov
 - bazni register hrani nalagalni naslov
 - mejni register hrani velikost naslovnega prostora
 - enostavno preslikovanje
 - enostavna zaščita
 - enostavno premeščanje



- Strojna podpora dinamičnega premeščanja
 - preslikovalna enota
 - bazni in mejni register
 - izvedba preslikovanja in preverjanja zaščite
 - pasti
 - prekoračitev mejnega registra
 - izvedba privilegiranih ukazov
 - v obeh primerih OS ukine proces
 - posebni strojni ukazi, ki jih uporablja le OS za manipulacijo z baznim in mejnim registrom in za nameščanje pasti
- Segmentacija in ostranjevanje
 - pomnilnik razdeljen na bloke velikosti 2^P
 - preslikovanje iz M bitnega navideznega naslovnega prostora v N bitni fizični naslovni prostor



- Segmentacija (mali OS)
 - bloki: segmenti
 - segmenti različnih velikosti
 - segmenti imajo nek pomen (koda, podatki...)
 - težavno upravljanje pomnilnika
 - zunanja fragmentacija
- Ostranjevanje (veliki OS)
 - bloki: strani in okvirji
 - strani so enake velikosti
 - strani nimajo posebnega pomena

- lažje upravljanje pomnilnika
- notranja fragmentacija

Ostranjevanje

- Stran
 - Blok v navideznem naslovnem prostoru
- Okvir strani
 - Blok v fizičnem naslovnem prostoru
 - okvir hrani poljubno stran
- Preslikovanje
 - navidezni naslov (naslov strani + odmik) preslikamo v

fizični naslov (naslov okvirja + odmik)

strani procesa A		strani procesa B	
-----		-----	
A0	Okvirji	----- B0	
A1	----- F0 /	B1	
A2	/ F1 /	B2	
A3	/ F2 povezave	B3	
A4	/ F3	-----\ B4	
A5	-----	\ B5	

- Preslikovanje s tabelo strani
 - 2^M -P deskriptjev v tabeli
- | P | 2^P | 2^M -P, M=32 | 2^M -P M = 64 |
|----|-------|----------------|-----------------|
| 12 | 4 KiB | 2^{20} | 2^{52} |
| 22 | 4 MiB | 2^{10} | 2^{42} |
- rešitvi: večnivojska tabela strani, invertirana tabela strani
 - Učinkovitost ostranjevanja
 - ena preslikava:
 - št. dostopov do pomnilnika = št. dostopov do tabel in podstrani + 1
 - rešitev:
 - TLB translation lookaside buffer
 - preslikovalnik predpomnilnik (SRAM v MMU enoti)
 - prostorska lokalnost: podatki so zelo blizu skupaj v pomnilniku (na isti strani)
 - ponoven dostop do istih podatkov
 - Naloge OS pri ostranjevanju
 - vodenje evidenc prostih okvirjev strani
 - upravljanje preslikovalnih tabel
 - stvaritev procesa - inicializacija naslovnega prostora
 - končanje procesa - sprostitve vseh zasedenih okvirjev
 - preslikava pomnilnika (mmap)
 - praznenje TLB ob preklopu procesa

Uporaba zunanjskega pomnilnika

- Pomnilniška hierarhija
 - notranji oz fizični pomnilnik: RAM
 - zunanji pomnilnik: trdi disk ali SSD
- Pomnilniški prekrivki (memory overlays)
 - celoten program je prevelik za pomnilnik, zato ga razdelimo na več kosov
 - programer sam poskrbi za sproščanje pomnilnika in nalaganje nove kode, ki prekrije staro
- Večprogramiranje
 - več procesov je hkrati naloženih v premajhnem pomnilniku: uporaba zunanjskega pomnilnika je nujna

- rešitev je **odlagalni prostor** (swap space), ki je del zunanjega pomnilnika, tam se izmenjujejo strani (pomnilnik -> disk in nazaj)
- Postopek naslavljanja podatka
 - zadetek v **TLB**
 - preveri če **VPN** obstaja v TLB
 - če obstaja, potem naslovi ustrezen del fizičnega pomnilnika
 - zgrešitev v TLB
 - pogledaj če je VPN v preslikovalni tabeli strani
 - če stran obstaja in je zapis veljaven, potem naslovi fizični pomnilnik
 - zgrešitev strani **page fault**, **page miss**
 - naloži stran iz zunanjega pomnilnika
 - osveži vnos v preslikovalni tabeli strani
 - ponovi strojni ukaz
- Kaj če je pomnilnik v celoti zaseden?
 - zgrešitev strani, ni prostora za novo, potem **page swap**
 - algoritmi za zamenjavo strani
 - **FIF** (furthest in future) je jasnovidni algoritem, optimalen
 - **FIFO** (first in, first out)
 - naključna zamenjava
 - **LFU** (least frequently used) moramo meriti frekvence
 - **LRU** (least recently used) moramo imeti časovne zaznamke
- Strojna pomoč
 - dodatni biti v vnosu tabele strani
 - **dirty bit** ali je bila stran spremenjena
 - **fixed bit** preslikava je fiksna, pusti na miru, dostop do V/I naprav
 - **access** ali je bila stran dostopana za branje / pisanje

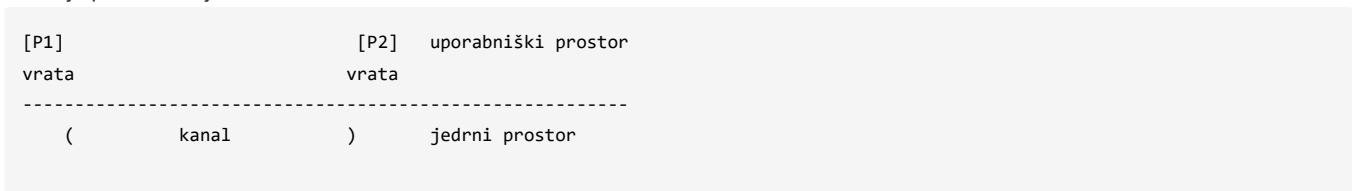
Medprocesna komunikacija

Medprocesna komunikacija

- Kaj to sploh je
 - Osnovni način komuniciranja med procesi (sočasni).
 - To je mehanizem OS-a, ki omogoča komuniciranje brez kršenja zaščite.
- Dva osnovna načina
 - prenašanje sporočil preko vtičnic, cevi itd
 - deljeni pomnilnik, pomnilniško preslikane datoteke
 - ostali načini pa so: argumenti programa, okoljske spremenljivke, izhodni statusi, stdin, stdout, datoteke, sinhronizacija procesov

Prenašanje sporočil

- Prenašanje sporočil
 - OS vzpostavi in sprost kanal, ga tudi upravlja, ter poskrbi za uspešen prenos in prejem
 - Kanal je ponavadi v jedu



- osnovne operacije za programski vmesnik so **pošiljanje**, **prejem**, včasih tudi **odgovor** na sporočilo
- Sinhronost komunikacije
 - sinhrona operacija
 - operacije blokirajo: pošiljanje blokira dokler ne odda v kanal in prejetje blokira dokler ne prejme sporočila
 - **zmenek/rendezvous**: kadar sta oba pošilji in prejmi sinhroni operaciji
 - asinhrona operacija
 - noben ne blokira: potrebujemo medpomnjenje sporočil

- Medpomnjenje
 - neka podatkovna struktura ki hrani poslana in še ne prejeta sporočila (vrste sporočil)
 - lahko ima **neomejeno** (pošiljatelj nikoli ne blokira), **omejeno** (pošiljatelj blokira pri polni vrsti sicer ne) ali **ničelno** kapaciteto (pošiljatelj vedno blokira dokler prejemnik ne prejme sporočila)

- Naslavljanje procesov

- neposredna komunikacija
 - procesi se morajo poznati
 - naslavljanje poteka preko **PID**
 - kanal se vzpostavi sam

```
send(pid, msg)
receive(pid, msg)
```

- posredna komunikacija
 - procesi komunicirajo preko **vmesnega objekta** - nabiralnika, vrat
 - več procesov lahko na enkrat komunicira
 - kanal moramo predhodno vzpostaviti

```
send(addr, msg)
receive(addr, msg)
```

- Lokacija nabiralnika v posredni komunikaciji

- Ta lahko živi v **naslovnem prostoru procesa**
 - nabiralnik živi dokler je živ proces, in se sprostí ko se proces konča
- Ali pa v **sistemskem prostoru izven procesa**
 - preživi proces
 - sporočila so na voljo tudi za kasnejše branje

- Prednost posredne komunikacije

- več pisalcev (procesov) lahko piše v isti nabiralnik
- več bralcev (procesov) lahko bere iz istega nabiralnika

```

          [P1]
          /
[P0] -> (  kanal  )
          \
          [P2]
```

- Učinkovitost

- komuniciranje zahteva sistemske klice
- **request-reply** : 4x prehod med uporabniškim in jedrnim načinom, 4x kopiranje podatkov
- **leno kopiranje** : kopiramo le naslov sporočila, kopiranje samega sporočila pa izvedemo kasneje, ko prejemnik prejme sporočilo

Sporočilo se kopira direktno iz pošiljateljevega naslovnega prostora v prejemnikovega.

- izboljššan **request-reply** : 4x prehod, 2x kopiranje podatkov

- Prenašanje sporočil je enostavno, saj OS skribo za stvaritev in upravljanje kanala, ter za medpomnjenje in sinhronizacijo.

- Problem **proizvajalec** - **porabnik**

- dana sta 2 ali več sočasnih procesov
- proizvajalec P proizvaja podatke in jih pošilja porabniku
- porabnik Q porablja podatke
- izzivi sočasnosti:
 - Ni znano kdo ima procesor: Procesor poljubno preklaplja med procesoma P in Q. Brez pravilne sinhronizacije lahko pride do izgube podatkov ali blokade (Deadlock).
 - Kdaj naj P proizvede nov podatek?: Odvisno od velikosti vrste (medpomnilnika). Če je vrsta polna, mora P počakati.
 - Ali Q lahko porabi podatek?: Samo, če je v vrsti vsaj eno sporočilo. Če je vrsta prazna, mora Q počakati.
 - Kako velika je vrsta sporočil med P in Q?: To je odvisno od kanala, lahko je ničelna, lahko je velikosti N (kjer je N največje število ki ga lahko sprejme), lahko pa je ∞ .

Deljeni pomnilnik

- Deljeni pomnilnik je pomnilnik ki je skupen dvema ali več procesom. Je del fizičnega pomnilnika preslikan v naslovni prostor procesov

fizični p

-----	-----	-----
P0		P1
-----	-----	-----

- Uporaba
 - vzpostavitev je storitev OS, dodajanje deljenega pomnilnika v naslovni prostor
 - komunikacija: neposredno branje in pisanje v deljeni pomnilnik
 - po koncu uporabe deljeni pomnilnik sprostimo
 - prednosti: hitrost, podatki so na voljo takoj
 - slabosti: potrebna explicitna sinhronizacija (ta se zgodi preko mehanizmov v knjižnici pthreads)

Primerjava

- Način prenašanja podatkov
 - prenašanje sporočil
 - kopiranje podatkov med različnimi naslovnimi prostori
 - hitra vzpostavitev, počasna komunikacija
 - čas kopiranja podatkov
 - deljeni pomnilnik
 - preslikava vsebine pomnilnika med naslovnimi prostori
 - manj hitra vzpostavitev, hitra komunikacija
 - čas vzpostavitve preslikave
 - enkratna vzpostavitev, večkratna uporaba

Medprocesna komunikacija API

Anonimne cevi

- Kaj je `cev(pipe)`
 - posredni sinhroni način
 - enosmerna komunikacija po principu `FIFO`, na eni strani pošiljamo v cev na drugi prihajajo ven
 - podatki se medpomnijo, to poskrbi OS

```
<--- ( cev ) <---
```

- Ustvarjanje cevi
 - ukaz `pipe()` ustvari in odpre cev, `fd[0]` predstavlja bralni konec cevi, `fd[1]` predstavlja pisalni konec cevi

```
int pipe(int fd[2])
```

```
branje                pisanje
fd[0] <--- ( cev ) <--- fd[1]
```

- Uporaba cevi
 - iz cevi beremo z

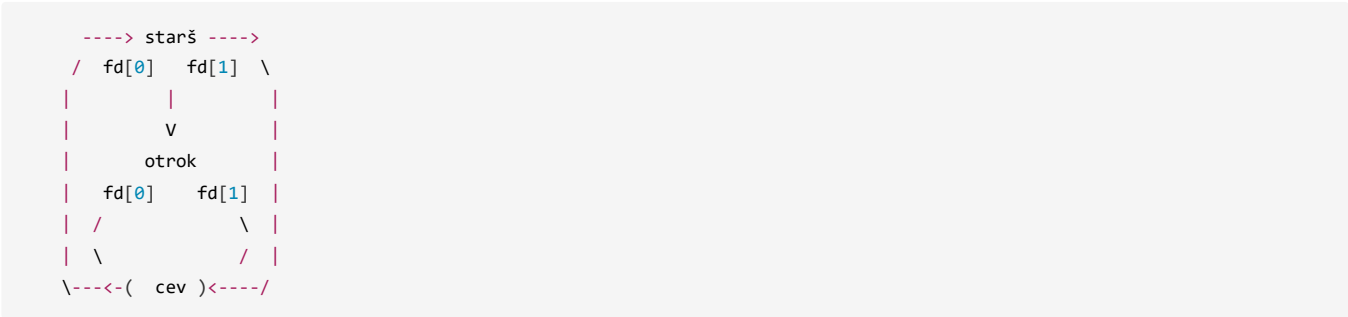
```
read(fd[0], ...)
```

- v cev pišemo z

```
write(fd[1], ...)
```

- oba klica lahko blokirata

- branje iz prazne cevi (prazen medpomnilnik)
- pisanje v polno cev (poln medpomnilnik)



- kombinacija je omejena na procese s skupnim prednikom (tipično: stariši in otroci in otroci med seboj)

```
[P0] -> ( cev 0 ) -> [P1] -> ( cev 1 ) -> [P2]
```

- o V lupini zaporedje ukazov ločimo z zankom `|`
- o lupina celoten cevovod pripravi sama
 - ustvari `pipe()`
 - ustvari otroke `fork()`
 - naredi preusmeritve `dup2()`
 - zažene ukaze v otrocih `exec()`
 - počaka da se vsi otroci končajo `waitpid()`

```
cat /etc/passwd | cut -d: -f7 | sort -u
```

```
[cat] -> ( cev 0 ) -> [cut] -> ( cev 1 ) -> [sort]
```

Imenovane cevi

- Imenovane cevi (named pipe): `fifo`
 - podobne anonimnim
 - poseben tip datoteke `pipe`: datoteke imajo imeniški vnos in ime, naslavljanje cevi poteka preko imena datoteke, zaščitni mehanizem kot za datoteke
 - neomejena komunikacija: proces, ki pozna ime cevi, jo lahko uporabi
- Ustvarjanje imenovanih cevi
 - sistemska klica:

- o ukaza:

```
mkfifo cev
mknod cev p
```

- Uporaba imenovane cevi
 - odpiranje in zapiranje:

- branje in pisanje:

```
read()
write()
```

Medprocesna komunikacija Cevi

Cevi

- Emosmerna komunikacija
 - enostavno pošiljanje in sprejemanje
 - pošiljamo v pisalni konec, sprejemamo na bralnem koncu
 - operaciji sinhroni

```
---> ( cev ) --->
```

- Medpomnjenje podatkov
 - princip FIFO – first in first out
 - za medpomnilnik poskrbi jedro OS

```
pošlji          prejmi
---\-----/---
   \         /
    ( medpomnilnik )
```

Anonimne cevi

- Posredno naslavljanje kanala
 - preko datotečnih deskriptorjev, anonimnost: ni imena datotek

```
// dva deskriptorja
int fd[2];

// pisanje          branje
fd[1] ---> ( cev ) ---> fd[0]
```

- Ustvarjanje in odpiranje cevi

```
// dva deskriptorja
int fd[2];

int pipe(int fd[2])
    fd[0] // bralni konec
    fd[1] // pisalni konec

// preverjanje napak
if (pipe(fd) == -1) {
    perror("pipe");
    exit(EXIT_FAILURE);
}
```

- Pisanje v cev, branje iz nje in zapiranje
 - pisanje v polno cev blokira in branje iz prazne tudi

```
write(fd[1], ...)
read(fd[0], ...)

close(fd[0])
close(fd[1])
```

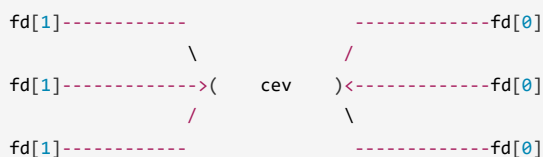
- potrebno je zapreti **VSE deskriptorje ki so povezani s cevjo**, sicer se cev ne zapre, ampak se sprosti le deskriptor.

Dedovanje cevi

- Dedovanje cevi: dedovanje datotečnih deskriptorjev: `fork()`
 - starš najprej ustvari cev: `pipe(fd)`, otroci dedujejo deskriptorje, torej tudi cevi



- Posledica dedovanja
 - bralni in pisalni konci cevi so dostopni preko več deskriptorjev



- pravilna inicializacija: zapremo vse deskriptorje, ki jih ne potrebujemo odprta naj ostaneta le dva deskriptorja
- Zaprtje zadnjega pisalnega konca cevi
 - predpostavimo pravilno inicializacijo cevi (en pisalni in en bralni deskriptor)
 - proces, ki piše v cev, zapre pisalni deskriptor
 - zato `read()` na bralni strani cevi vrne EOF (red tem, se do konca prebere medpomnilnik)
- Zaprtje zadnjega bralnega konca cevi
 - predpostavimo pravilno inicializacijo cevi (en pisalni in en bralni deskriptor)
 - proces, ki bere iz cevi, zapre bralni deskriptor
 - zato `write()` na pisalni strani cevi (povzroči pošiljanje signala `SIGPIPE` pisalnemu procesu)

Cevovodi

- Cevovodi v lupini: estav ukazov, ločenih z znakom `|`
 - stopnja cevovoda: št povezanih ukazov, št povezanih procesov

```
cat /etc/passwd | cut -d: -f7 | sort -u
```

- Priprva cevovoda:

- starš (lupina)

```
ustvarjanje cevi: pipe()
ustvarjanje otrok: fork()
zapiranje odvečnih cevi: close()
čakanje otrok: waitpid()
```

- otroci (posamezne stopnje)

```
preusmeritve : dup2()
zapiranje deskriptorjev: close()
zagon ukazov: exec()
```

- diagrami na prosojnicah

- Izhodni status cevovoda v lupini je enak izhodnemu statusu zadnje stopnje

- Izhodni statusi vseh stopenj cevovoda v lupini

```
echo ${PIPESTATUS[@]}
```

Imenovane cevi

- Posredno naslavljanja kanala

- preko datotečnega imena
- poseben tip datoteke – cev (pipe)
- kdor pozna ime cevi, jo lahko odpre
- velja mehanizem zaščite datotek: r ... branje cevi, w ... pisanje cevi, x ... se ignorira
- Ustvarjanje imenovane cevi
 - sistemski klic

```
mkfifo(path, mode)
```

- ukaz v lupini

```
mkfifo cev
mkfifo cev p
```

- Uporaba

```
odpiranje: open()
posebnost: odpiranje za branje blokira, dokler se ne odpre tudi za pisanje
branje: read()
pisanje: write()
zapiranje: close()
```

Niti

Niti

- Glavni nalogi procesov sta
 - **lastništvo** oz zaščita virov
 - **izvajanje kode**
 - Zakaj nebi obeh nalog ločili

Procesi naj skrbijo za lastništvo virov, niti pa za izvajanje kode.
- Enonitnost in večnitnost (single vs multi-threading)
 - en proces: ena nit na proces (Microsoft DOS)
 - en proces: več niti na proces (Java VM)
 - več procesov: ena nit na proces (nekateri Unixi)
 - več procesov: več niti na proces (moderni Windowsi, moderni Unixi)
- Kaj je nit
 - skrbi za izvajanje kode
 - ima svoj **izvajalni kontekst** (stanje, registre in sklad)
 - pripada nekemu procesu: vse niti istega procesa imajo **isti naslovni prostor**, delijo si tudi ostale vire (npr odprte datoteke)

Zakaj niti

- Glavne prednosti večnitnosti so
 - ustvarjanje in končanje niti je hitrejšo kot procesov (brez inicializacije naslovnega prostora)
 - preklapljanje med nitmi je učinkovitejše kot preklap med procesi, saj ni potrebno preklopiti naslovnega prostora in izprazniti TLB-ja.
 - ker si niti delijo isti pomnilnik, je medprocesna komunikacija zelo učinkovita
 - boljši izkoristek več-procesorskega sistema
 - boljše izkoriščanje virov (manjša poraba pomnilnika, saj si ga niti delijo)
 - vsaka nit poskrbi za svoje opravilo, asinhrono procesiranje
- Primeri uporabe
 - spletni brskalnik: nit za prikaz strani, nit za nalaganje podatkov
 - urejevalnik besedila: nit za prikaz, nit za LSP, nit za preverjanje slovnice itd.
 - spletni strežnik: glavna sprejemna nit, nit za vsako posamezno zahtevo na strežniku
 - večnitnost v jedru OS: niti za rokovalje prekinitev, nit za menjavo strani itd.

Izvedba niti

- Izvedba niti na **jedrnem nivoju**

- OS razume in skrbi za niti 😊
- podpora nitim preko sistemskih klicev
- razvrščevalnik razvršča niti tako kot procese
- niti seveda lahko izkoriščajo več procesorjev (jeder procesorja)
- sistemski klici ki blokirajo, niso problematični, saj blokira le ustrezna nit
- Izvedba niti na **uporabniškem nivoju**
 - upravljanje niti v uporabniškem programu prek knjičnice za nitenje
 - razvrščanje niti je lahko prilagojeno aplikaciji
 - niti lahko tečejo v kateremkoli OS
 - hitrejši preklon med nitmi (vse se zgodi na uporabniškem nivoju)
- OS se niti na uporabniškem nivoju **ne zaveda**
 - če ena nit blokira, potem blokira celoten proces (torej vse niti)
 - vse niti so izvedene v sklopu enega samega procesa
 - niti na uporabniškem nivoju so nezmožne izkoriščati več procesorjev
- Rešitev preslikava uporabniških niti v jedrne
 - več-v-eno: več uporabniških niti se preslika v eno jedrno nit
 - več-v-več: več uporabniških niti se preslika v manj ali enako število jedrnih niti
 - ena-v-eno: ena uporabniška nit se preslika v eno jedrno nit (Windows, Linux)
- Večnitni naslovni prostor

```
0kb                                     128kB
-----
| koda | kopica | prosto | sklad 2 | prosto | sklad 1 |
-----
```

- Mehanizem za izvedbo niti je **deskriptor niti** (kontrolni blok v jedru)
 - to je podatkovna struktura v jedru, ki hrani podatke o niti
 - registri, stanje niti, sklad, PC
 - souporaba pomnilnika: navidezni pomnilnik -> enaka preslikava iz navideznega v fizični pomnilnik
- Preklop niti
 - podoben preklonu procesa, s tem da ni zamenjave naslovnega prostora in ni praznjenja TLB-ja
- Stvaritev niti

```
operacija | uporabniške niti | jedrne niti | procesi
-----
null fork | 34 | 948 | 11300
signal wait | 37 | 441 | 1840

null fork : stvaritev, razvrščanje, izvedba, končanje klica prazne funkcije
signal wait : čakanje na izvedbo ustvarjene niti / procesa
```

Nitni izzivi

- Kaj naredi `fork()` v večnitnem procesu?
 - Klonira **samo klicočo nit**. Novi (otroški) proces po `fork()` vsebuje le eno nit, in sicer kopijo niti, ki je izvedla `fork()`. Druge niti iz starševskega procesa se ne prenesejo.
- Kaj naredi `exec()` v večnitnem procesu?
 - **Nadomesti celoten proces** z novim programom. Klic `exec()` zamenja celoten naslovni prostor procesa, zato izginejo vse niti, stari program pa se nadomesti z novim. Po uspešnem `exec()` se začne izvajati novi program kot en sam proces z začetno nitjo.
- Kaj pa zagon novega programa v otroku z `fork() & exec()` ?
 - Fork ustvari otroški proces, v otroki obstaja samo nit ki je izvedla fork. Exec pa nadomesti celoten naslovni prostor otroškega procesa z novim programom. Po uspešnem `exec()` se začne izvajati nov program, stanje prejšnjega programa in njegove niti se ne ohranijo.
- Pokočanje niti (končanje niti preden se opravilo zaključi)
 - **asinhrono končanje** nit se konča takoj (`Thread.stop()` starinsko)
 - **odloženo končanje** nit se konča ko zaključi trenutno opravilo
 - sproščanje zasedenih virov: kdo sprosti vire, če se nit asihrono konča? Ponavadi to sprosti OS, ko zaključi trenutni proces, sicer pa mora to storiti aplikacija, ki nit uporablja torej programer sam.
- Rokovanje signalov: Katera nit rokuje signal?
 - Tista, ki je vzrok za pošiljanje signala, za signale ki nastanejo zaradi dejanj te niti (npr izjeme)

- Vse niti procesa? Ne, tipično.
- Samo izbrane niti? Morda odvisno je sicer od vrste signala.
- Izbrana nit rokuje vse signale? Da, to je možni in pogosto uporabljeno -> posebna izbrana nit rokuje vse asinhronne signale.
- Število nit in bazen niti (thread pool)
 - Recimo da imamo strežnik, ki ima veliko zahtev in vsak zahteva uporablja eno nit. To lahko povzroči preobremenitev sistema: Rešitev je *thread pool*, ki ima končno število delavskih niti (*worker thread*), ki se po končanju opravila ponovno uporabijo. Te niti so vnaprej pripravljene, za hitro delovanje.

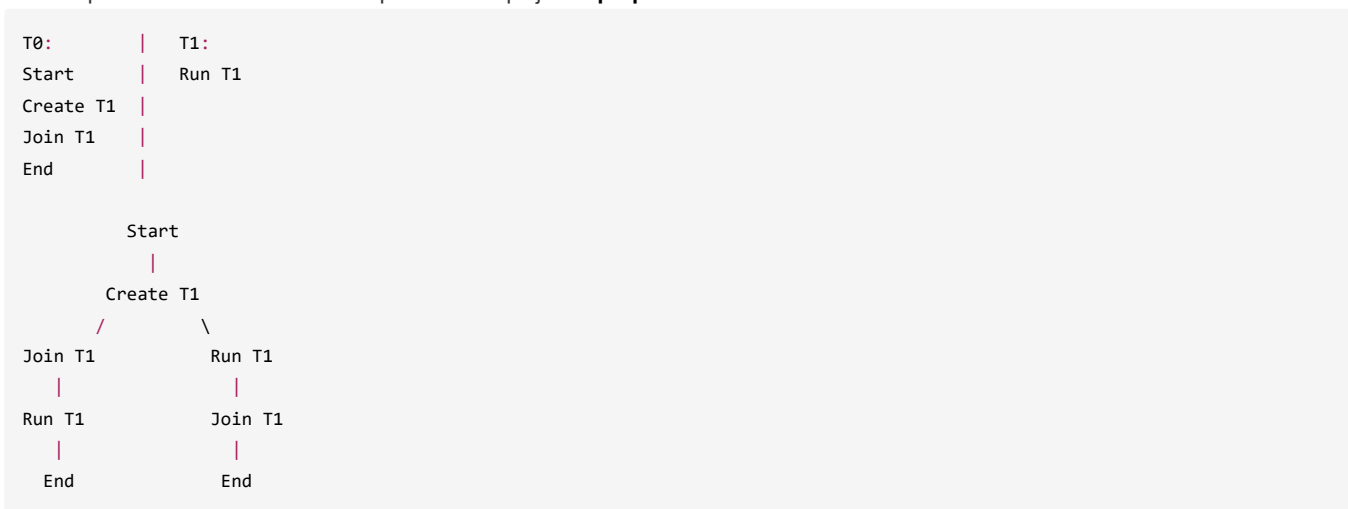
Sočasnost

Sočasnost

- Osnovni pojmi
 - sočasnost (concurrency): prekrivanje obstoja večih procesov, občutek hkratnega izvajanja
 - vzporednost (parallelism): dejansko vzporedno izvajanje (več procesorjev)
 - porazdeljenost (distributed): izvajanje več procesov v več vozliščih omrežja
- Sočasnost in vzporednost
 - sočasnost (brez vzporednosti): večopravilnost, prepletanje izvajanja ukazov
 - sočasnost z vzporednostjo: več procesorski sistem, prekrivanje ukazov
 - obe vrsti povzročata enake težave in rešitve
 - deljenje globalnih virov
 - smrtni objem (**deadlock**)
 - težavno debugiranje
- Kje se pojavlja sočasnost
 - izvajanje več aplikacij sočasno: multiprogramiranje nastalo zaradi tega
 - sočasno izvajanje ene aplikacije: pogled na aplikacijo kot množico vzporednih procesov
- Odnosi med procesi
 - **tekmovalnost**: procesi se ne zavedajo drug drugega, tekmujejo za vire
 - **sodelovanje**: preko medprocesne komunikacije ali deljenih virov, procesi se zavedajo drug drugega, vzpostavijo komunikacijo z mehanizmi, ki jih nudi OS

Sočasno izvajanje ukazov

- Prepletanje izvajanja
 - multiprogramiranje: več procesov, en procesor
 - izvedba posameznih ukazov različnih procesov se poljubno **prepleta**



- Prekrivanje izvajanja
 - multiprocesiranje: več procesov, več procesorjev
 - izvedba ukazov različnih procesov se poljubno **prekriva**

Souporaba vira

- Skupni / deljeni vir (shared data)

- vir, ki ga lahko uporablja dva ali več procesorjev (npr podatki oz. spremenljivka v večnitnem izvajanju)
- Souporaba skupnega vira
 - sočasna uporaba vira s strani več procesorjev
 - rezultat izvajanja procesov
- Primer
 - dve niti / procesa P1 in P2 se sočasno izvajata
 - uporabljata skupni vir oz. spremenljivko, ki se začne z vrednostjo `i=0` :
 - eden inkrementira spremenljivko `i=i+1`
 - eden dekrementira spremenljivko `i=i-1`
 - opazujemo rezultat izvajanja: dejanska vrednost spremenljivke po izvajanju je odvisna od konkretne arhitekture procesorja in od kod v katero se prevede

```
load R, i
add R, 1
store R, i

load R, i
sub R, 1
store R, i
```

- Kako se prepleta izvajanje kode

```
// poljubno prepletanje izvajanja ukazov
P1      P2
load R, i
        load R, i
add R, 1
        sub R, 1
store R, i
        store R, i
```

```
// poljubno prekrivanje izvajanja ukazov
P1      P2
load R, i
add R, 1 load R, i
store R, i sub R, 1
        store R, i
```

Tvegano stanje

- Tvegano stanje oz. **Race condition**
 - rezultat souporabe skupnega vira je odvisen od prepletanja / prekrivanja izvajajočih se ukazov
 - **tveganim** stanjem se želimo izogniti, zato je koda, ki povzroči tvegano stanje napačna (bug)
 - izvajanje na različnih platformah, lahko različno pogosto vodi v tvegano stanje (debugging sočasnih programov je težaven)
- Primeri tveganih stanj

```
// Dva procesa en vir
P: x = 1
Q: x = 2
x = ?
```

```
// N procesov en vir
init: x = 0
Pi: x = x + 1
x = ?
```

```
// Dva procesa dva vira
init: x = 1, y = 2
P: x = x + y
Q: y = x + y
x = ?, y = ?
```

```
// Dva procesa dva vira
init: x = 0, y = 0
P: x = x + 1, y = y - 1
Q: x = x - 1, y = y + 1
x = ?, y = ?
```

Kritični odsek

- Kritični odsek je del programske kode, kjer se sočasno uporablja skupni vir. Sočasna uporaba skupnega vira vodi v tvegano stanje.
- Vzajemno izključevanje (mutual exclusion)
 - to je zagotovilo, da se znotraj KO (kritičnega odseka) sočasno nahaja največ en proces
 - **vstop** in **izstop** iz kritičnega odseka (ostalim procesom ne dovolimo vstopa, dokler prvotni proces ne izstopi)
- Uporaba kritičnega odseka
 - vstop v KO je možen le če noben drug proces ni v KO. Stanje procesa ni pomembno, pomembna je vrednost programskega števca
 - po vstopu v KO ostali procesi ne morejo v KO, kritični odsek je zaseden
 - izstop iz KO je vedno možen, po izstopu je KO prost, poljuben proces lako vstopi vanj (lahko znova isti)

Izzivi kritičnih odsekov

- **Stradanje** (starvation)
 - proces, ki želi vstopiti v KO ne pride na vrsto ali predolgo čaka, saj ga vedno nekdo prehit, ki ima višjo prioriteto
 - zato procesom ki dlje časa ne dobijo vira vsake toliko časa povečamo prioriteto
- **Smrtni objem** oz (deadlock)
 - dva ali več procesov ne more nadaljevati izvajanja, ker ciklično čakajo drug na drugega (čakajo na vire ki jih držijo drugi procesi) (primer: 4 enakovredne ceste in 4 avtomobili)
 - procesa P in Q oba potrebujeta vira A in B za nadaljevanje izvajanja, vendar pri tem proces P prej pridobi vir A in proces Q prej pridobi vir B

Proces P	ProcesQ
- acquire A	- acquire B
- acquire B	- acquire A
- ...	- ...
- release B	- release A
- release A	- release B

- Reševanje iz smrtnega objema
 - ukinjanje nekega procesa v ciklu deadlocka, tako da se viri ki jih ta proces drži sprostijo
 - prevzem vira ki si ga lasti nek proces
 - obnavljanje stanja (**rollback**): periodično hranimo stanje procesa, če se zgodi deadlock, ponovno zaženemo proces
- **Živi objem** oz (livelock)
 - Podoben smrtnemu, le da procesi ne čakajo, temveč menjavajo stanja brez napredovanja
- Izzivi
 - enakopravnost, poštenost (kdo vpliva na izbor procesa za vstop v KO)
 - omejeno čakanje (izogibanje stradanju)
 - učinkovitost (hiter izbor procesa)
 - poljubno prepletanje ali prekrivanje ukazov

Vzajemno izključevanje

Ključavnica

- To je osnovi mehanizem za zagotavljanje vzajemnega izključevanja.
 - Vzajemno izključevanje pomeni, da se v kritičnem odseku sočasno nahaja le en proces.
 - Ključavnica ščiti odsek (vstop in izstop iz odseka)
- Poskus izvedbe Ključavnice

```

fun lock_init(lock) is
  lock = 0

fun lock_enter(lock) is
  while lock == 1 do nothing
  lock = 1

fun lock_exit(lock) is
  lock = 0

```

- Kaj vse je tu narobe?
 - Imamo sočasno preverjanje stanja ključavnice in ločenost nastavljanja stanja od preverjanja
 - rešitev? **Atomarnost** celotne operacije

```

atomic fun lock_enter(lock) is
  while lock == 1 do nothing
  lock = 1

```

- Kaj je **Atomarna** operacija?
 - to je zaporedje enega ali večih ukazov, ki je s programskega vidika nedeljeno zaporedje. Izvede se kot celota ali pa se sploh ne izvede.
 - izolacija od drugih sočasnih procesov: Noben drug proces ne more prekiniti operacije ali imeti vpogleda v vmesno stanje operacije
 - vzajemno izključevanje: izvede se le ena atomarna operacija na enkrat (več procesorjev)
 - obvoz predpomnilnika (več procesorjev): zaradi usklajevanja vrednosti operanda, počasnejša operacija
- Izziv
 - strojnih ukazov `lock_enter` in `lock_exit` ni, običajni strojni ukazi ponavadi niso atomični
 - Kako zagotoviti atomičnost ključnih operacij? Glej naprej.

Strojna izvedba

- Onemogočanje prekinitev
 - vstop v KO: **onemogočimo** prekinitev
 - izstop iz KO: **omogočimo** prekinitev
 - Slabosti tega
 - uporaba privilegiranih ukazov (x86): `STI` (omogoči) in `CLI` (onemogoči)
 - to onemogoči večopravilnost: če se kritični odsek zacikla se zacikla cel sistem
 - to ne deluje na večprocesorski arhitekturi: na ostalih procesorjih so prekinitev še vedno mogoče
- Namenski strojni ukazi
 - **atomični** strojni ukazi
 - načelo ključavnice: vstop (zaklepanje), izstop (odklepanje)
 - **vstop**: dokler je zaklenjeno čakaj (vrteče čakanje - *busy waiting* / *spin waiting*)
 - **izstop**: enostavno prirejanje

```

// init ključavnica, skupna spremenljivka
flag=0

// vstop v KO: zaklepanje ključavnice
fun enter() is ... odvisno od ukaza

// izstop iz KO: odklepanje ključavnice
fun leave() is
  flag = 0

```

- ukaz `test & set`
 - če je zaklenjeno potem vrni `true`
 - sicer zakleni in vrni `false`

```

atomic instruction test_and_set(ref val) is
  if val then return true
  val = true
  return false
fun enter() is
  while test_and_set(&flag) do nothing

```

- ukaz `compare & swap`
 - posplošitev `test & set`
 - če je trenutna vrednost enaka testni vrednosti, potem jo zamenjaj z novo vrednostjo
 - vedno vrne staro vrednost

```
atomic instruc,on compare_and_swap(ref val, testval, newval) is
    oldval = val
    if val == testval then val = newval
    return oldval

fun enter() is
    while compare_and_swap(&flag, false, true) do nothing
```

- ukaz `exchange`
 - zamenjava dveh vrednosti

```
atomic instruc,on exchange(ref a, ref b) is
    t = a
    a = b
    b = t

fun enter() is
    key = true
    do exchange(&flag, &key) while key
```

- ukaz `fetch & add`
 - poveča vrednost, vrne staro vrednost

```
atomic instruction fetch_and_add(ref val) is
    oldval = val
    val += 1
    return oldval
```

- Ključavnica s prepustnico

```
fun init() is
    ticket = turn = 0

fun enter() is
    myturn = fetch_and_add(&ticket)
    while turn != myturn do nothing

fun exit() is
    fetch_and_add(turn)
```

- Prednosti namenskih strojnih ukazov
 - rešitev deluje tudi na večprocesorskih sistemih
 - enostavnost izvedbe
 - podpora večim kritičnim odsekom (več zastavic)
- Slabosti namenskih strojnih ukazov
 - vrteče čakanje zapravlja procesorski čas (*busy waiting*)
 - možno stradanje: izbira vstopnega procesa je "naključna" oz odvisna od razvrščevalnika
 - možen smrtni objem: P1 dobi vir, prekine ga P2 z večjo prioriteto, ki gre v vrteče čakanje, saj ne dobi vira

Programska izveba

- Dekkerjev algoritem
 - prvi algoritem za reševanje KO, dva procesa si predajata prednost, vrteče čakanje

```

me = 0
you = 1 - me
shared entering = [false, false]
shared turn = you

fun enter() is
  entering[me] = true
  while entering[you] do
    if turn == you then
      entering[me] = false
      while turn == you do nothing
      entering[me] = true
    endif
  endwhile

fun leave() is
  turn = you
  entering[me] = false

```

• Petersonov algoritem

- vzajemno izključevanje z vidika procesa P-me
- če je P-me v KO, potem je entering[me] = true
- za P-you velja ena od možnosti
 - ne želi vstopiti: entering[you] = false
 - želi vstopiti: entering[you] = true in je predal prednost: turn = me, zato čaka
 - želi vstopiti: entering[you] = true in še ni predal prednosti: turn = you, zato še ni prišel do while zanke

me = 0 oz. 1	-> P0: me = 0, P1: me = 1
you = 1 - me	-> P0: you = 1, P1: you = 0
shared entering = [false, false]	-> deljena spr., začetek: nobeden ne želi vstopiti
shared turn = you	-> deljena spr., na potezi je drugi proces
fun enter() is	
entering[me] = true	-> želim vstopiti
turn = you	-> dam 9 prednost
while entering[you] and	-> dokler želiš 9 vstopiti in
turn == you do nothing	-> je tvoja poteza, jaz čakam
fun leave() is	
entering[me] = false	-> ne želim več vstopiti

• Lamportov pekarski algoritem

- za N procesov princip oštevilčenih listkov

```

shared taking = [false, false, ..., false]
shared number = [0, 0, ..., 0]
me = zaporedna št. procesa

fun enter() is
    taking[me] = true           // dodeljevanje številke
    number[me] = max(number) + 1 // največja dodeljena številka + 1
    taking[me] = false

    // počakaj, da nit j dobi številko
    // če j vstopa in če ima manjšo številko, potem čakaj
    for j = 0 to N - 1 do
        while taking[j] do nothing
        while number[j] != 0 and
            (number[j] < number[me] or number[j] == number[me] and j < me)
            do nothing

fun leave() is
    number[me] = 0

```

Učinkovitost ključavnic

- Mere zmogljivosti
 - **zakasintveni čas** (latency): To je čas za pridobitev proste ključavnice. Optimalno: takoj izvedemo atomično operacijo.
 - **čakalni čas** (delay): Čas za pridobitev ravnokar sproščene ključavnice. Optimalno: takoj pridobimo.
 - **tekmovanje** (contention): Promet na pomnilniškem vodilu zaradi *atomične operacije* in *zagotavljanje koherentnosti predpomnilnika*. Optimalno: nič.
- Izvedbe ključavnic
 - osnovna različica

```

fun enter() is
    while test_and_set(&flag) do nothing

```

- test + test and set

```

fun enter() is
    while flag or test_and_set(&flag) do nothing

flag      ... // testiranje v predpomnilniku, vrtenje
test_and_set ... // atomična operacija, dostop do glavnega pomnilnika

```

- + zakasnitev

```

fun enter() is
    while flag or test_and_set(&flag) do
        while flag do nothing
        delay()
    end

```

- Se komu vrti? 🤪
 - **vrteča ključavnica** (spinlock): porablja procesorski čas za čakanje

```

fun enter() is
    while test_and_set(&flag) do nothing

```

- uporabimo `yield()` (če ga imamo)

```

fun enter() is
    while test_and_set(&flag) do yield()

```

- Bi kdo raje spal? 🇸🇮
 - namesto vrtečega čakanja naj nit raje blokira, kdaj se zbudi?
 - nit se zbudi ko druga nit **izstopi** iz KO in sprostí ključavnico, v funkciji `exit()` sproži signal za sprebujanje (nastavi zastavico).


```

fun enter() is
    while test_and_set(&flag) do
        wait_for_flag_to_be_changed()

fun exit(ref lock) is
    lock = 0
    wake_up_some_waiting_thread()

```

Sinhronizacija

Ključavnica

- Ključavnica (lock, **mutex** - mutual exclusion)
 - mehanizem **zaklepanja**: dve stanji (odklenjeno in zaklenjeno)
 - nit, ki zaklene ključavnico si jo lasti (jo drži): dvojno zaklepanje ključavnice iz strani iste niti lahko vodi v smrtni objem ali kako drugo napako
 - le **ena** nit si lahko sočasno lasti ključavnico
 - nit, ki zaklene ključavnico jo lahko tudi odklene (odklepanje ključavnice, ki jo drži druga nit ni definirano)
- Uporaba
 - vzajemno izključevanje na kritičnih odsekih
 - operacija **zakleni** (lock, acquire): če je ključavnica odklenjena jo zaklene, sicer čaka na odklenjeno ključavnico
 - operacija **odkleni** (unlock, release): odklene (ista nit) ključavnico, ki je zaklenjena

```

pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

pthread_mutex_lock(&m);
// kritični odsek
pthread_mutex_unlock(&m);

```

Problem proizvajalci-porabniki

- Proizvajalci-porabniki
 - dana sta dva ali več sočasnih procesov
 - dve vrsti procesov: proizvajalec **P** proizvaja podatke, porabnik **Q** jih porablja
 - komunikacija poteka preko souporabe vrste: pride do **tevaganega stanja** pri sočasnem dostopu do vrste. Vrsta ima omejeno ali neomejeno kapaciteto.
- Rešitev z zaklepanjem

```

pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
queue_t q = ...;

// Proizvajalec P
while true do
    data = produce_data()
    pthread_mutex_lock(&m)
    q.enqueue(data)
    pthread_mutex_unlock(&m)
done

// Porabnik Q
while true do
    pthread_mutex_lock(&m)
    if not queue.empty then
        data = q.dequeue(data)
    pthread_mutex_unlock(&m)
    if data then
        consume_data(data)
done

```

- A se da bolje, moti nas namreč neprestano preverjanje pogoja ali je vrsta prazna.
- Rešitev je uporaba pogojne spremenljivke

- To je mehanizem **obveščanja**. Opazovalni pogoji je treba eksplicitno preverjati (lahko gre za več pogojev).
- Množica čakajočih niti: to so niti ki čakajo na izpolnjenost pogoja.
- nit, ki obvesti čakajoče niti o izpolnjenosti pogoja: Vsaj ena čakajoča nit se zbudi in nadaljuje izvajanje.
- To pripomore k optimizaciji delovanja procesorja (ni vrtečega čakanja).
- Operacija **čakaj** (wait):
 - sprosti pripadajočo ključavnico
 - trenutna nit se postavi v čakano množico
 - blokira trenutno nit (prekop na drugo pripravljeno nit)
 - ko se nit spet zbudi, ponovno dobi ključavnico
- Operacija **obvesti** (signal, notify)
 - zbudi eno nit iz čakalne množice
 - namen: obvestilo o izpolnjenem pogoju
- Operacija **obvestiVse** (broadcast, notifyAll)
 - zbudi vse niti iz čakalne množice
 - namen: kadar je več izpolnjenih pogojev
- Rešitev z zaklepanjem in pogojno spremenljivko

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;
```

```
void* make(void *arg) {
    // obdelava pred izpolnitvijo pogoja
    ...
    pthread_mutex_lock(&m);
    // update shared state
    if (POGOJ)
        pthread_cond_signal(&c)
    pthread_mutex_unlock(&m);
}
```

```
void* watch(void *arg) {
    pthread_mutex_lock(&m);
    while (not POGOJ) do
        pthread_cond_wait(&c, &m)
    // update shared state
    pthread_mutex_unlock(&m);

    // obdelava po izpolnitvi pogoja
    ...
}
```

- Proizvajalec in porabnik

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;
queue_t q = ...;
```

```
// Proizvajalec P
data = produce_data()
pthread_mutex_lock(&m)
q.enqueue(data)
pthread_cond_signal(c)
pthread_mutex_unlock(&m)
```

```
// Porabnik Q
while true do
    pthread_mutex_lock(&m)
    while q.empty() do
        pthread_cond_wait(m, c)
    q.dequeue(data)
    pthread_mutex_unlock(&m)
    consume_data(data)
done
```

- Omejena kapaciteta vrste
 - Kaj naj naredi proizvajalec če je vrsta polna?
 - Počaka (zaspi) na pogojni spremenljiki, dokler se v vrsti ne naredi prostor. (`pthread_cond_wait`)
 - Kdo zbudi spečega proizvajalca?
 - Porabnik, ko odstrani element iz polne vrste in s tem sprostí prostor (`pthread_cond_signal`).
 - Koliko pogojnih spremenljivk potrebujemo?
 - Običajno 2: `not_full` proizvajalci čakajo, ko je vrsta polna in `not_empty` porabniki čakajo, ko je vrsta prazna.
 - Katere obveščanje (signal, broadcast) uporabimo?
 - Običajno `pthread_cond_signal()`, ker je dovolj zbuditi eno čakajočo nit.
 - `pthread_cond_broadcast()` uporabimo le, če želimo zbuditi vse čakajoče niti.

Problem bralci-pisalci

- Bralci- pisalci
 - več procesov sočasno dostopa do podatkov
 - bralci le berejo, pisalci podatke tudi spreminjajo

```
# bralcev | # pisalcev | možne težave | komentar
-----
>0      | 0      | NE      | sočasno branje ni težava
0       | 1      | NE      | le en pisalec ni težava
0       | >1     | DA      | sočasno pisanje je težava
>0      | >0     | DA      | sočasno branje in pisanje je težava
```

- Rešitev z osnovnim zaklepanjem
 - branje obdamo s ključavnico
 - pisanje obdamo s ključavnico
 - slabost: onemogočimo sočasno branje več bralcev. Se da bolje?

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t r = PTHREAD_COND_INITIALIZER;
pthread_cond_t w = PTHREAD_COND_INITIALIZER;
volatile int counter = 0; // volatile pomeni ne optimiziraj prevajalnik!
```

```
// BRALEC
pthread_mutex_lock(&m)
    while counter == -1 do
        wait(m, r)
    counter++
pthread_mutex_unlock(&m)

// branje podatkov

pthread_mutex_lock(&m)
    counter--
    if counter == 0 then
        signal(w)
pthread_mutex_unlock(&m)
```

```
// PISALEC
pthread_mutex_lock(&m)
    while counter != 0 do
        wait(m, w)
        counter = -1
pthread_mutex_unlock(&m)

// pisanje podatkov

pthread_mutex_lock(&m)
    counter = 0
    broadcast(r)
    signal(w)
pthread_mutex_unlock(&m)
```

Bralno-pisalna ključavnica

- **Izključujoče zaklepanje** (exclusive, write lock)
 - le ena nit jo lahko sočasno zaklene
 - onemogoča souporabo vira drugim nitim
 - uporaba: pisanje ne da bi komu pokvarili branje
- **Delno zaklepanje** (shared, read lock)
 - več niti jo lahko sočasno "deljeno" zaklene (več deljenih ključavnic)
 - onemogoča souporabo izključujoče ključavnice
 - uporaba: enovito/celovito branje

```
pthread_rwlock_t lock = PTHREAD_RWLOCK_INITIALIZER;

// bralni odsek
pthread_rwlock_rdlock(lock);
// enovito branje
pthread_rwlock_unlock(lock);

// pisalni odsek
pthread_rwlock_wrlock(lock);
// pisanje
pthread_rwlock_unlock(lock);
```

- **Svetovalna ključavnica** (advisory lock)
 - neodvisnost operacije od ključavnice
 - dejansko branje/pisanje se ne ozira na stanje ključavnic
 - programer mora sam poskrbeti za ustrezno uporabo ključavnic pri bralno/pisalnih odsekih programa
- **Obvezna ključavnica** (mandatory lock)
 - odvisnost operacije od ključavnice
 - OS poskrbi, da branje/pisanje upošteva stanje ključavnic
- Izveba v POSIX
 - zaklepanje je vezano na `inode` (ni vezano na ime datoteke ali datotečni deskriptor)
 - vsak proces si lahko lasti eno ali več ključavnic
 - ključavnice se **ne** dedujejo: `fork()`
 - sistemski klic

```
int flock(fd, tip)
fd: LOCK_SH (shared), LOCK_EX (exclusive), LOCK_UN (unlock), LOCK_NB (non-blocking)
```

- shell ukaz: `flock -n fd in lockfile`

Semafor

- Izzivi
 - zahtevna uporaba ključavnic in pogojnih spremenljivk
 - potrebujemo tako ključavnice kot pogojne spremenljivke
 - rešitev: mehanizmi, ki združujejo delovanje

- Semafor
 - semafor združuje števec ključavnico in pogojno spremenljivko
 - **pozitivna** vrednost števca (vključno z 0) predstavlja odprt semafor in prost prehod procesa v KO
 - **negativna** vrednost števca pa predstavlja zaprt semafor, prehod v KO ni možen, proces čaka na odprtje semaforja
- Uporaba semaforja
 - operacija **wait** (tudi *test, down*): dekrementira števec, če je semafor zaprt, potem gre proces v čakalno vrsto semaforja
 - operacija **signal** (tudi *increment, up*): inkrementira števec in zbudi morebitni čakajoči proces iz semaforjeve vrste
 - Kako zagotoviti atomičnost obeh operacij -> strojni ukazi in kratko vrteče čakanje
 - Izvedba in uporaba

```
// priprava
sem: Semaphore
init(sem, 1)
...
// zaklepanje
wait(sem)
// kritični odsek
signal(sem)
```

```
struct Semaphore is
  count: Int
  queue: Queue

fun init(sem, value) is
  sem.value = value
  sem.queue = []

atomic fun wait(sem) is
  sem.count -= 1
  if sem.count < 0 then
    queue.enqueue(this process)
    make_waiting(this process)

atomic fun signal(sem) is
  sem.count += 1
  if sem.count <= 0 then
    process = queue.dequeue()
    make_ready(this process)
```

- POSIX API - Semafor
 - primer

```
// tip semafor
sem_t sem;

// inicializacija semaforja
sem_init(sem_t *sem, int pshared, int count)
                        ^      ^
                        |      |
                    način souporabe  Začetna vrednost

// operacija wait
sem_wait(sem_t *sem);

// operacija signal
sem_post(sem_t *sem);
```

- Dvojiški semafor

```
// inicializacija semaforja
sem_init(sem_t *sem, int pshared, 1)
```

- Števni semafor

```
// inicializacija semaforja
sem_init(sem_t *sem, int pshared, N)
```

Monitor

- Monitor
 - je višje nivojski konstrukt (Java, C#..)
 - vstop in izstop v kritični odsek generira prevajalnik
- vsebnik
 - eksplicitno specificira souporabljeni vir
 - vsebuje lokalne spremenljivke in funkcije
 - na enkrat se lahko znotraj monitorja izvaja le ena funkcija, četudi je več procesov
 - medsebojno izključevanje izvajanja funkcij
- primer

```
monitor BancniRacun is
    stanje: Int

sync fun poloziDenar(polog) is
    stanje += polog

sync fun dvigniDenar(dvig) is
    stanje -= dvig

sync fun obresti(p) is
    stanje *= (1 + p / 100)
```

- Monitor in **Java**
 - vsak objekt lahko uporabimo kot monitor

```
class BancniRacun {
    int stanje;

    synchronized void poloziDenar(int polog) { stanje += polog; }

    synchronized void dvigniDenar(int dvig) { stanje -= dvig; }

    synchronized void obresti(p) { stanje *= (1 + p / 100); }

    void ostaleFunkcije() {}
}
```

- `synchronized` funkcije in bloki

```
synchronized void f() {
    ...
}

f() {
    synchronized (this) {
        ...
    }
}
```

Ostali mehanizmi

- Ostali mehanizmi
 - Pregrada (*barrier*): Ideja delovanja je obratna od semaforjev. Semafor dopušča vstop N procesom, pregrada pa za vstop zahteva N procesov.
 - serializers: definicija prioritet
 - path expressions: regularni izrazi za definicijo usklajenega obnašanja
 - wait-free sync: optimistično, read-copy-update ključavnica

Proizvajalci-porabniki

- Rešitve:
 - sinhrona sporočila: če sta *send* in *receive* operaciji sinhroni, potem je problem avtomatsko razrešen
 - s semaforji: 3 semaforji : medsebojno izključevanje, sporočanje prazne vrste, sporočanje prazne vrste
 - monitorji: 3 monitorji : podobno semaforjem

Lačni filozofi

- Problem:
 - Kitajski filozofi sedijo za okroglo mizo, vsak si s svojim sosedom deli eno palčko.
 - Torej ima vsak 2 palčki, levo si deli z levim sosedom, desno pa z desnim.
 - Vsaka palčka predstavlja delejni vir. Ko filozof ni lačen razmišlja in odloži obe palčki. Ko pa filozof je, pa uporablja obe palčki.
 - Izziv: smrtni objem in stradanje.
- Rešitve:
 - Centralna avtoriteta, ki odreja prehranjevanje. Filozof lahko vzame samo obe palčki hkrati.
 - asimetrična rešitev: lihi filozofi vzamejo najprej levo palčko, nato desno, sodi filozofi pa najprej desno in nato levo.

Virtualizacija

- Virtualizacija: mehanizem ki nekaj ustvari navidezno
 - navidezna naprava (procesor, pomnilnik)
 - navidezni stroj (celotni sistem, strojna oprema)
 - Primer: navidezni disk oz. datotečni sistem
 - uporbaniku enovito dostopen preko klasičnih sistemskih klicev (open, read, wrtie, close)
 - v ozadju pa se navidezni disk preslika na dejanski sistem npr: (več diskov, porazdeljeni datotečni sistem, jedrni datotečni sistem)
 - Abstrakcija in virtualizacija
 - podobna koncepta, oba nudita nek vmesnik (do nekega) sistema
 - abstrakcija skriva podrobnosti (kje in kako je datoteka dejansko shranjena)
 - virtualizacija lahko podrobnosti pusti odprte (navidezni procesor je lahko ravno tako zahteven za programiranje kot dejanski)
 - Virtualizacijska programska oprema
 - vključuje upravljanje in konfiguracijo, ter nudi virtualizacijski vmesnik navideznim strojem
- navidezni stroj (VM)
nadzornik (VMM)
- **gostitelj** (host): dejanski sistem, ki poganja virtualizacijsko programsko opremo
 - **gost** (guest): sistem ki ga virtualiziramo, je nek navidezni stroj
- Vrste virtualizacije:
 - procesni navidezni stroj: Virtualizacijska programska oprema, neko izvajalno okolje **runtime**, ki uporablja domači OS in strojno opremo. Omogoča prenosljivost aplikacij.
 - sistemski navidezni stroj
 - vsebniki

Lastnosti virtualizacije

- Replikacija virov
 - Videz da je več virov (naprav), kot jih dejansko je. Omogoča sočasnost navideznih strojev, OS in aplikacij.
- Varnost in neodvisnost
 - Navidezni stroji so med seboj **neodvisni** in **izolirani**
 - Gostiteljski OS ščiti pred aplikacijami (dva nivoja zaščite)
 - Zaščita nadzornika pred gostitelji (dodaten virtualizacijski nivo zaščite)
- Prilagodljivost in enostavno upravljanje
 - večje razpoložljivost in zasnesljivost (selitev strojev med virtualizacijskimi platformami)
 - zmanjševanje stroškov (več navideznih strojev na enem dejanskem stroju)
 - lažje upravljanje (navideznih) strojev (namestitev OS, debugiranje, podpora starim OS)
- Slabosti
 - manjša učinkovitost, saj na dejanskem stroju teče več navideznih

- upravljanje dejanskega stroja je bolj zahtevno (namestitvev in konfiguracija nadzornika)
- virtualizacija lahko ni popolna (dostopnost dejanskih naprav v navideznem stroju je lahko težavno)

Sistemska virtualizacija

- Sistemski navidezni stroj
 - virtualizira celoten sistem oz. strojno opremo (procesor, pomnilnik, naprave)
 - stara ideja -> zakaj se ni razširila že v 60-ih? Zato ker veliki osrednji računalniki niso bili razširjeni, mali računalniki so bili poceni. H temu se postopoma spet vračamo z AI.
 - zakaj se je razširila šele sedaj? -> Podatkovni centri so postali ogromni, nudijo boljšo izkoriščenost strežnikov, omogočajo varno souporabo strežnikov Administracija tolikšne količine strežnikov je zahtevna, zraven pa pridejo tudi visoki stroški obratovanja strežnikov.
- Virtualizacija **tipa 1** (bare metal - hypervisor based)
 - nadzornik navideznih naprav VM monitor: `hipervisor` (*hypervisor*) -> Teče **neposredno** na strojni opremi
 - servisni navidezni stroj: Omogoča konfiguracijo in upravljanje sistema. Privilegirane operacija, dostop do strojne opreme, poganja gonilnike za strojno opremo.

```

-----
|  OS  |  OS  |  OS  |
| s VM |  VM  |  VM  |
|      |hipervisor|
|      |strojna oprema|
-----

```

- Xen (opensource), VMware ESXi (open API), Microsoft Hyper-V
- Virtualizacija **tipa 2** (host-OS-based)
 - temelji na gostiteljskem OS, primeri: VMware Workstation, Oracle VirtualBox ...

```

-----
|  OS  |  OS  |  OS  |-----|  aplikacija |
| s VM |  VM  |  VM  | app |    |      OS      |
-----|  nadzornik  |
|      |gost. OS + VM modul|    |      OS      |
|      |strojna oprema  |    | strojna oprema |
-----|-----|

```

- Polna virtualizacija (full virtualization)
 - preslikava vseh pomembnih funkcij dejanskega stroja v funkcije navideznih strojev: (celoten nabor strojnih ukazov, prekinitve, vhodno/izhodne operacije, dostop do pomnilnika)
 - gostujoči OS se ne zaveda virtualizacije in deluje enako kot v ne virtualiziranem okolju
- Paravirtualizacija
 - učinkovitost za ceno spremembe gostujočega OS
 - gostujoči OS ve, da se izvaja virtualizirano
 - OS ne izvaja privilegiranih ukazov, ampak eksplicitno kliče nadzornika (nadzorni klici, hiper klici (hyper calls))
 - originalno Xen, sedaj XenSource

Izvedba virtualizacije

- Souporaba / deljenje časa (time sharing / partitioning)
 - souporaba procesorja (deljenje procesorskega časa)
 - souporaba naprav (časovno deljenje uporabe)
- Souporaba / deljenje prostora (space sharing / partitioning)
 - deljenje fizičnega pomnilnika (vsak proces uporablja del pomnilnika)
- Podpora strojne opreme
 - Intel VT-x in AMD-V: *root* in *non-root* način delovanja, preklap med gostujočim OS in nadzornikom. VM-exit in VM-entry.
 - obroč -1: *root* nivo (nadzornik vseh strojev)
 - obroč 0: *non-root* nivo 0 (gostujoči OS)
 - obroč 3: *non-root* nivo 3 (aplikacije)

Virtualizacija procesorja

- Virtualizacija procesorja

- strojni ukazi se izvedejo neposredno na procesorju: nepriviligirani ukazi (učinkovitost izvajanja), priviligirani ukazi (past, ki jo rokuje nadzornik)
- izveba priviligiranih ukazov: emulira jih nadzornik, tako kot to pričakuje gostujoči OS
- Težave pri izvedbi virtualizacije
 - primer Intel x86, pred 2005 -> brez *root* načina.
 - obroči 0: nadzornik, 1: OS, 3: aplikacije.
 - 17 priviligiranih ukazov ni prožilo pasti v obroču 1+, le potihoma niso uspeli, nadzornik torej ni naredil nič in OS je mislil da je vse OK.
- Prevajanje (binary translation)
 - v kodi nadomesti problematične ukaze
 - Cilj: brez sprememb v gostujočem OS, dinamično dvojiško prevajanje (pregledovanje blokov kode, ki se bodo izvedli, nadomeščanje priviligiranih ukazov)
- Polna virtualizacija:

[navidezni naslov]	-->	[navidezni fizični naslov]	-->	[dejanski fizični naslov]
	preslikovalna tabela		preslikovalna tabela	
	s katero upravlja		s katero upravlja	
	gostujoči OS		nadzornik	

- Dvojno preslikovanje
 - programsko in strojno (MMU, TLB) ... neučinkovito
- Senčna preslikovalna tabela
 - na podlagi preslikovalnih tabel gostujočih OS
 - nadzornik vodi preslikovalno tabelo, ki slika iz navideznih naslovov v dejanske fizične naslove (MMU)
- Paravirtualizacija
 - s preslikovalnimi tabelami upravlja nadzornik
 - gostujoči OS se zaveda virtualizacije, zato lahko prilagodi uporabo pomnilnika in pri nadzorniku eksplicitno registrira svoje preslikovalne tabele
- Izzivi
 - veliko različnih naprav, manj standardizacije obnašanja naprav (primerjava s CPU in pomnilnikom)
- Pristopi
 - passthrough model:
 - VMM konfigurira dostop do naprav in tako pridobi ekskluzivni in neposredni dostop do naprave (zaobide nadzornika)
 - souporaba naprave je zelo težavna
 - migracija navideznih strojev je težavna
 - hypervisor-direct model
 - nadzornik prestreže vse dostope do naprave
 - emulira odziv naprave
 - nadzorni stroj je ločen od naprave
 - migracija in souporaba je lažja
 - hypervisor mora omogočati gonilnike
 - Split-device driver model
 - dostop do naprave je razdeljen med *front-end* gonilnik v navideznem stroju in *back-end* gonilnik v servisenem navideznem stroju